

AI Search Assistant

System Working and Technical Documentation

AI-Powered Document Question Answering Platform using Retrieval-Augmented Generation (RAG)

Project Documentation

Technology Domain:

Artificial Intelligence | RAG | NLP | Semantic Search | Web Development

Project Introduction

AI Search Assistant is an AI-powered document question-answering application designed to help users search and understand information from uploaded PDF documents. Users can upload PDFs related to different technologies, and the system processes the document content through a Retrieval-Augmented Generation (RAG) pipeline. When a user asks a question, the system searches the processed PDF content, retrieves the most relevant information, and provides that information to the Gemini model to generate a document-based answer. The application also maintains PDF metadata and user-wise document information, helping organize uploaded files and support future multi-user functionality.

AI Search Assistant – System Working and Technical Documentation

Section 1 — Technologies Used and Their Role

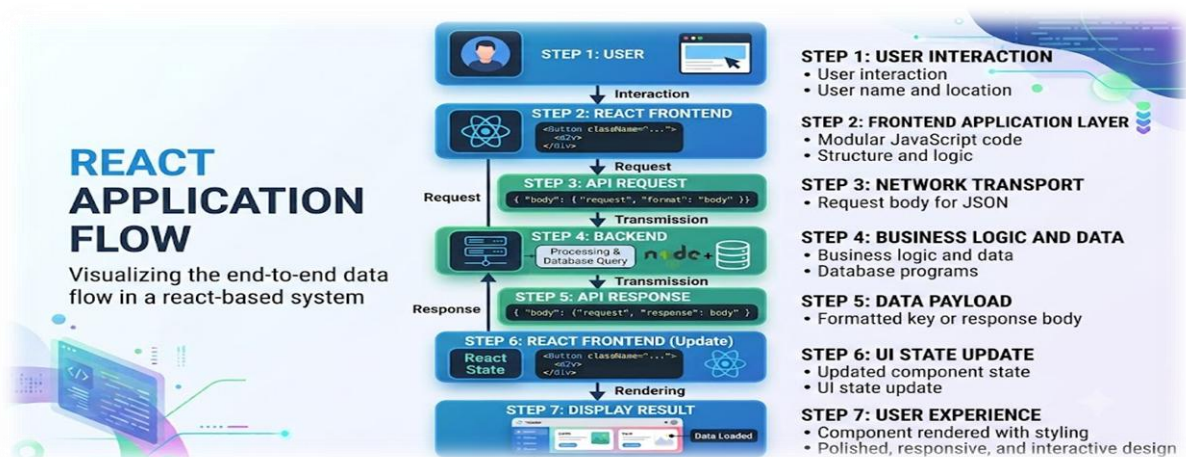
This section describes the technologies used in the project, why they are used, what work they perform, and where they are used in the project workflow

1. React.js

Used For: Frontend Development

React.js is used to develop the frontend of the application. It provides the user interface through which users can register, log in, upload PDFs, view their uploaded PDF list, and ask questions to the AI assistant. React communicates with the backend through APIs and displays the data received from the backend.

Workflow:

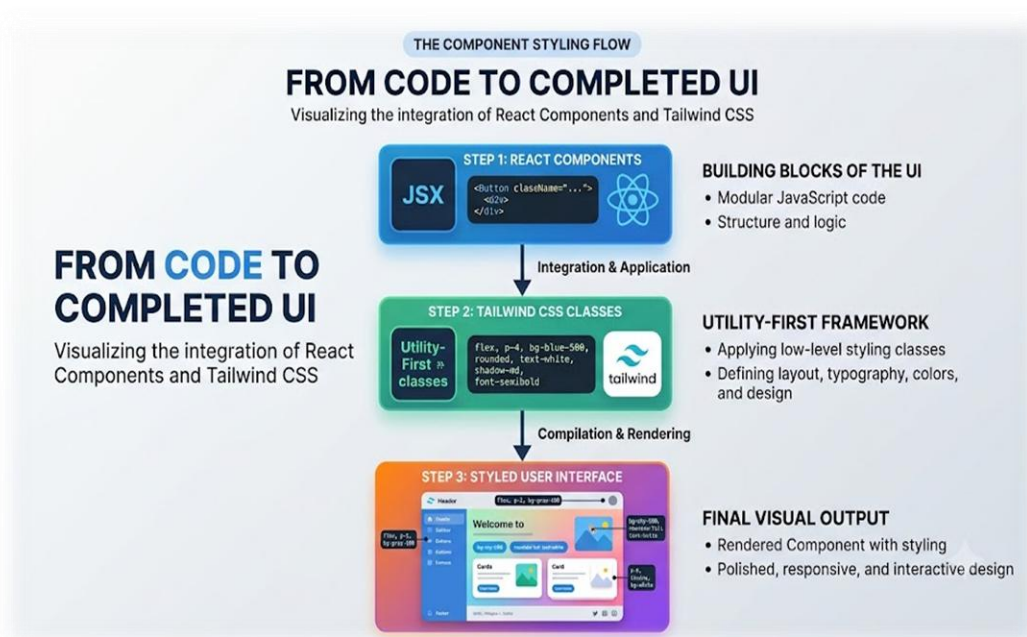


2. Tailwind CSS

Used For: User Interface Styling

Tailwind CSS is used to design and style the React frontend. It is used for layouts, buttons, forms, PDF cards, navigation components, chat interfaces, and responsive pages. It helps maintain a consistent design across the application without writing large amounts of custom CSS.

Workflow:

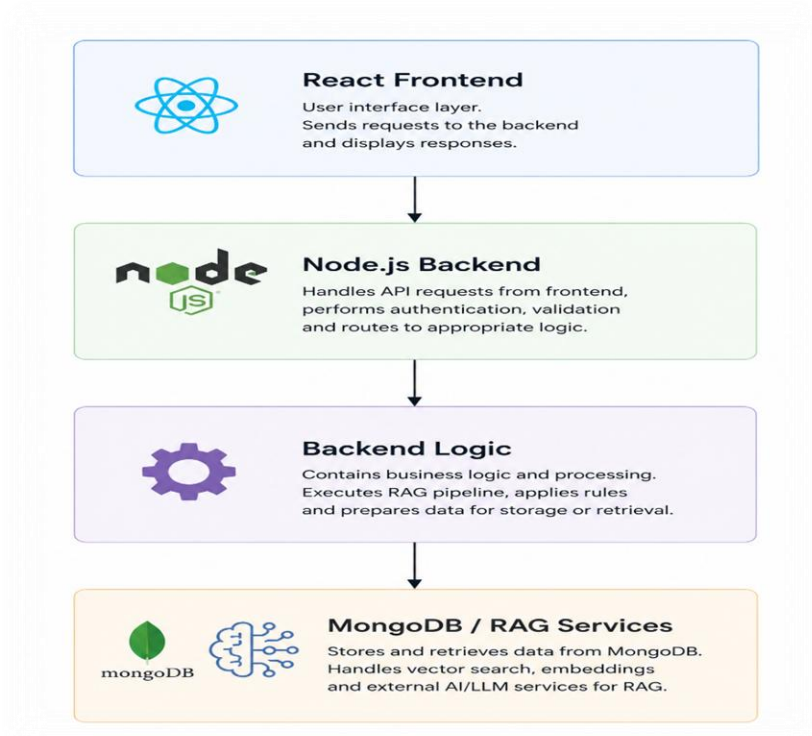


3. Node.js

Used For: Backend Runtime

Node.js is used to run the backend application. It handles incoming requests from the React frontend and executes the backend logic. It also allows the application to communicate with MongoDB and other services used in the project.

Workflow:

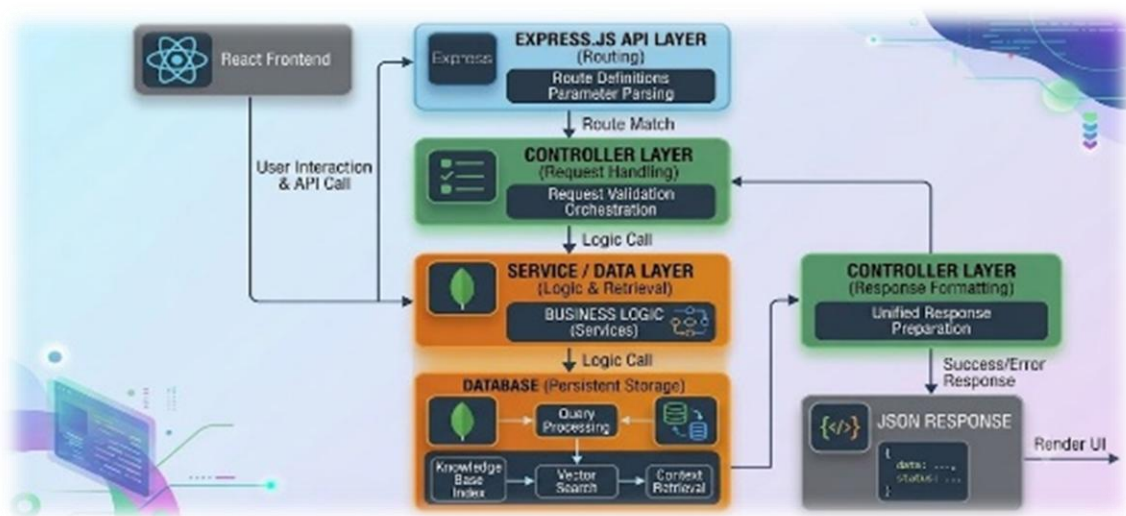


4. Express.js

Used For: API Development

Express.js is used with Node.js to create and manage the backend APIs. It handles routes such as user registration, login, PDF upload, PDF list, PDF deletion, and chat requests. When the frontend sends a request, Express receives the request, calls the required controller or service, and sends the response back to the frontend.

Workflow:



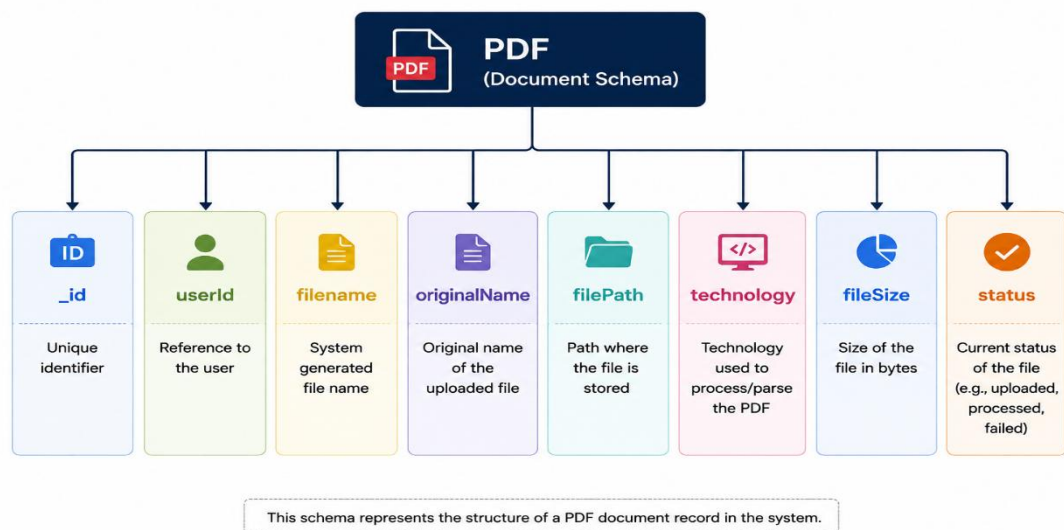
5. MongoDB

Used For: Application Data and PDF Metadata Storage

MongoDB is used as the main application database. It stores structured information such as users, PDF metadata, technologies, folders, and chat-related data.

For uploaded PDFs, MongoDB stores information such as the PDF ID, user ID, filename, original filename, file path, technology, file size, and processing status.

For example:

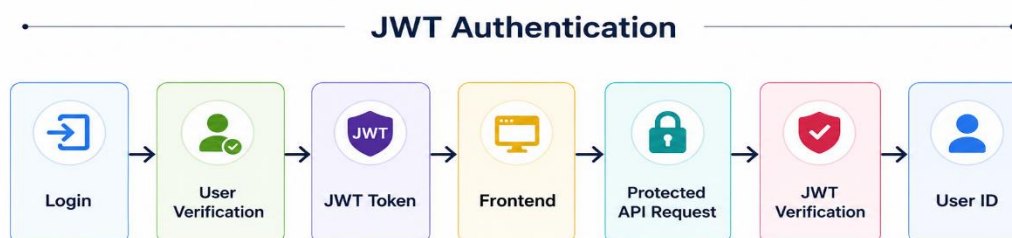


6. JWT Authentication

Used For: User Authentication and Identification

JWT is used to authenticate users and identify the logged-in user. After successful login, the backend generates a JWT token. The frontend sends this token with protected API requests. The backend verifies the token and obtains the user's ID.

Workflow:



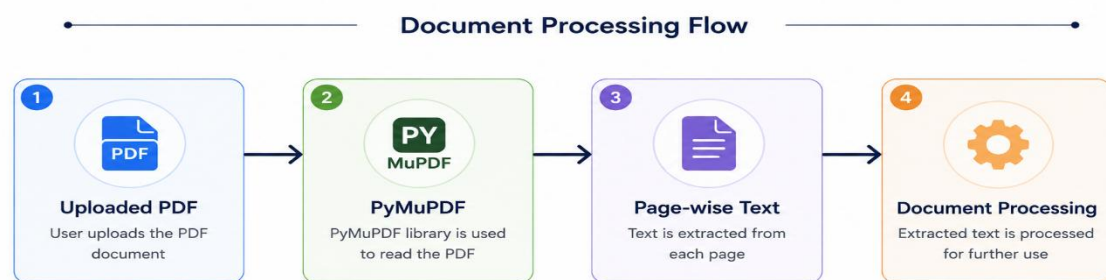
The user ID can then be used to access user-specific PDFs and other user-related data.

7. PyMuPDF

Used For: PDF Text Extraction

PyMuPDF is used to read uploaded PDF files and extract their text page by page. The extracted text is then passed to the document processing pipeline. Page information is maintained so that the system can identify the source page when generating an answer.

Workflow:

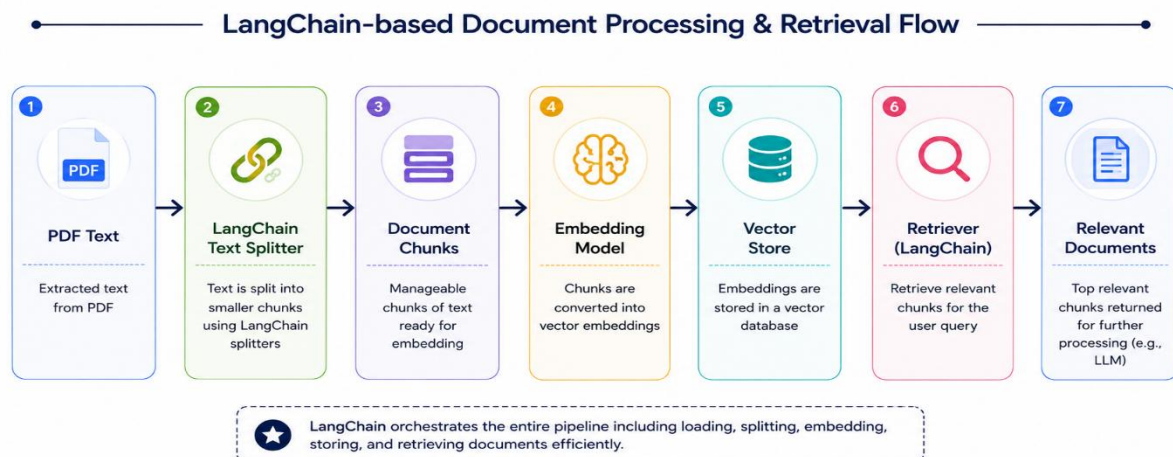


8. LangChain

Used For: RAG and Document Processing

LangChain is used to manage important parts of the RAG pipeline, such as document processing, text chunking, retrieval, and prompt handling. It connects the different components of the RAG pipeline so that PDF content can be processed and retrieved efficiently.

Workflow:

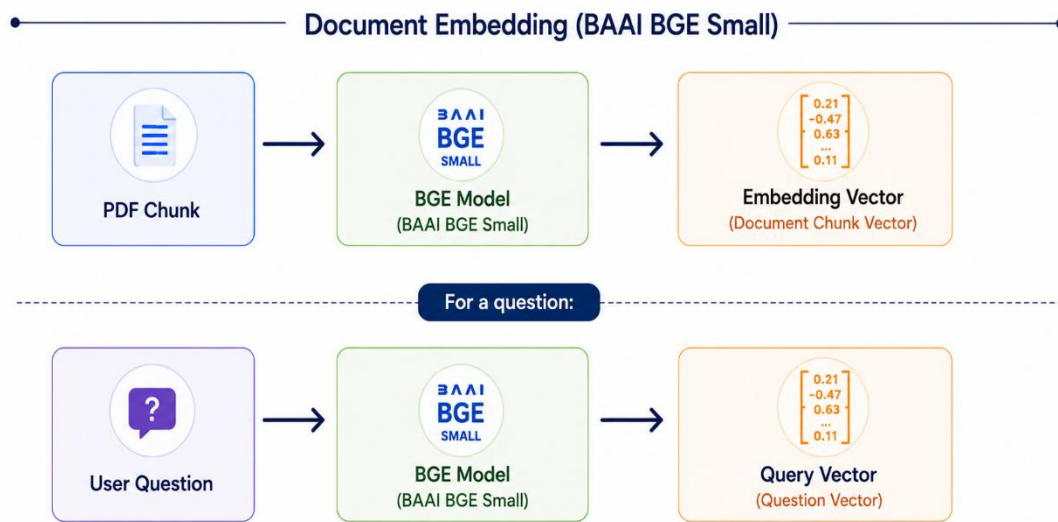


9. BAAI BGE Small

Used For: Embedding Generation

BAAI BGE Small is used to convert PDF text chunks into numerical vectors called embeddings. It is also used to convert the user's question into a query vector. These vectors allow the system to compare the meaning of the question with the meaning of the stored document chunks.

Workflow:

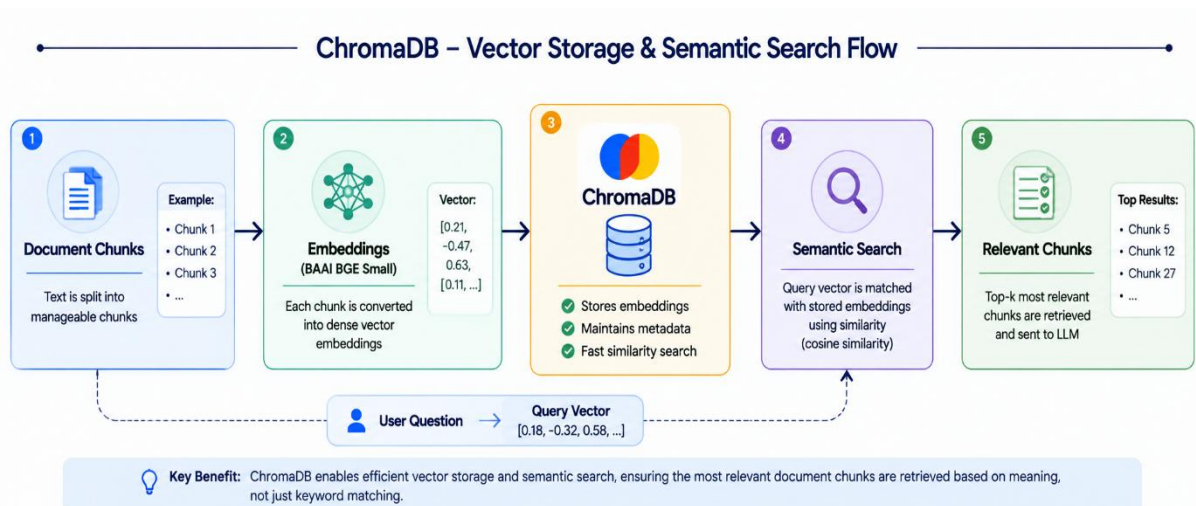


10. ChromaDB

Used For: Vector Storage and Semantic Retrieval

ChromaDB is used as the vector database for the RAG pipeline. It stores document chunks, embeddings, and related metadata. When the user asks a question, the query embedding is compared with the stored embeddings to find the most relevant document chunks.

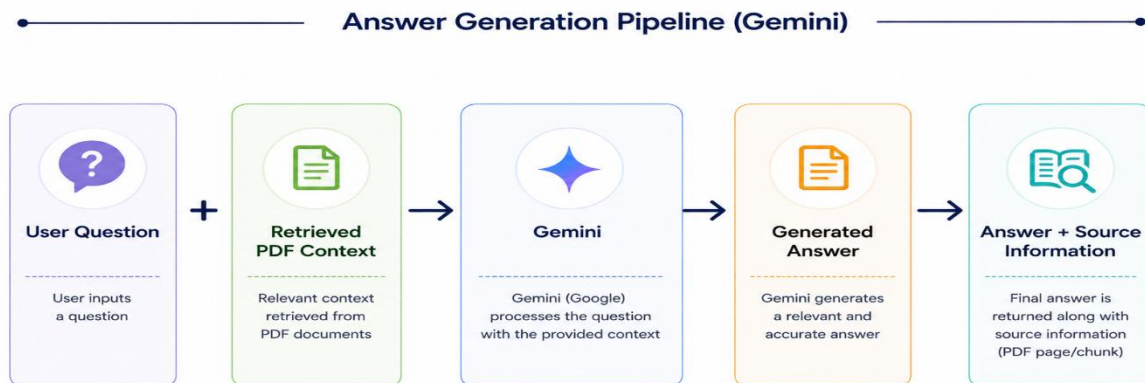
Workflow:



11. Gemini

Used For: Final Answer Generation

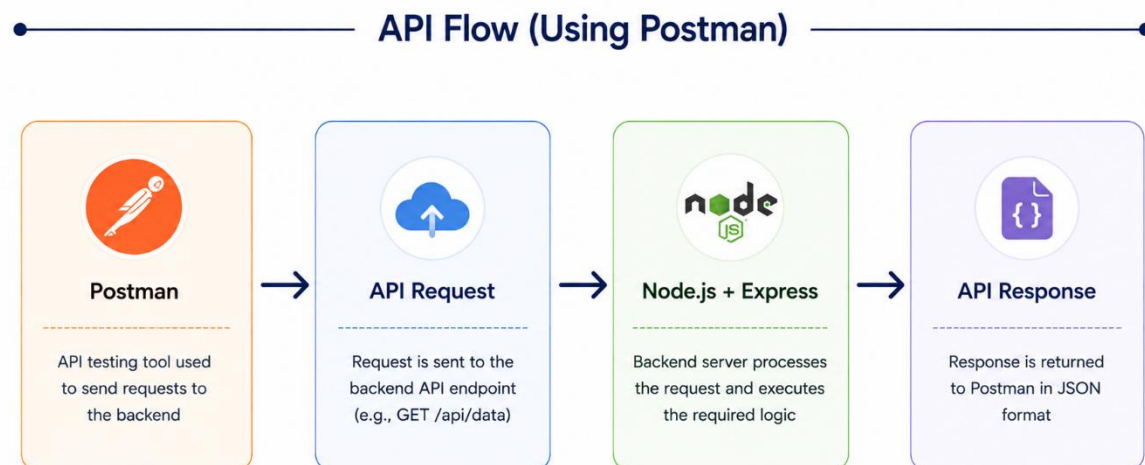
Gemini is used as the Large Language Model for generating the final answer. The user's question and the relevant chunks retrieved from ChromaDB are provided to Gemini as context. Gemini uses this context to generate the answer.



12. Postman

Used For: API Testing

Postman is used to test the backend APIs during development. APIs such as login, PDF upload, PDF list, and chat can be tested using Postman before connecting them to the React frontend.



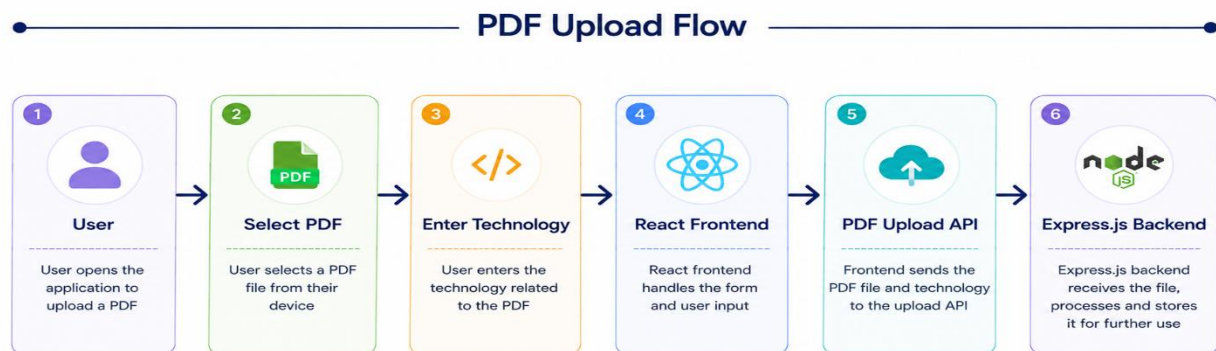
Section 2 – PDF Upload, Storage and User-PDF Mapping

This section explains how a PDF is uploaded by the user, how its file information is stored in the system, how the PDF is connected with the user, and how the uploaded document becomes available for further RAG processing.

2.1 PDF Upload

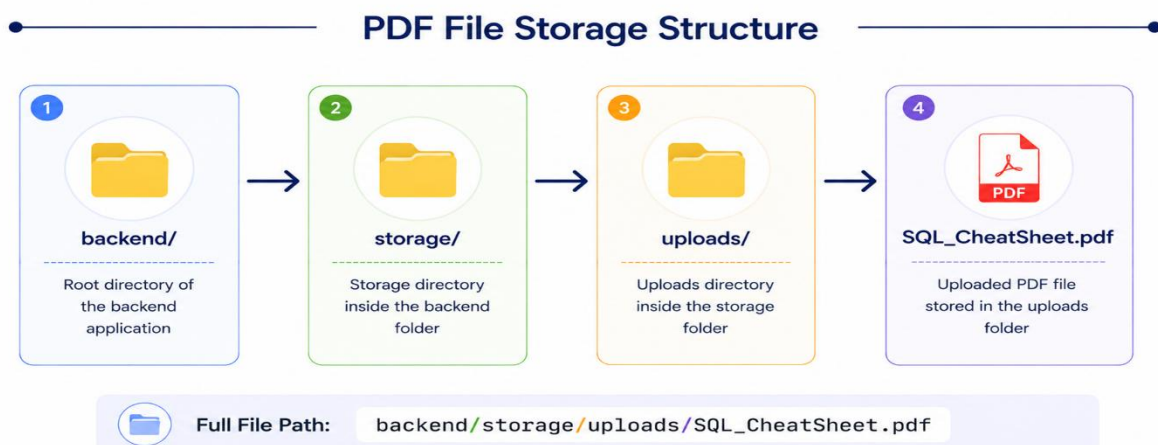
The PDF upload process starts from the React frontend. The user selects a PDF file and provides the required technology information. The frontend sends the PDF and its related information to the backend through the PDF upload API.

The Express.js backend receives the request and checks whether a PDF file and the required technology information are available. If the required information is valid, the backend creates a PDF record in MongoDB.

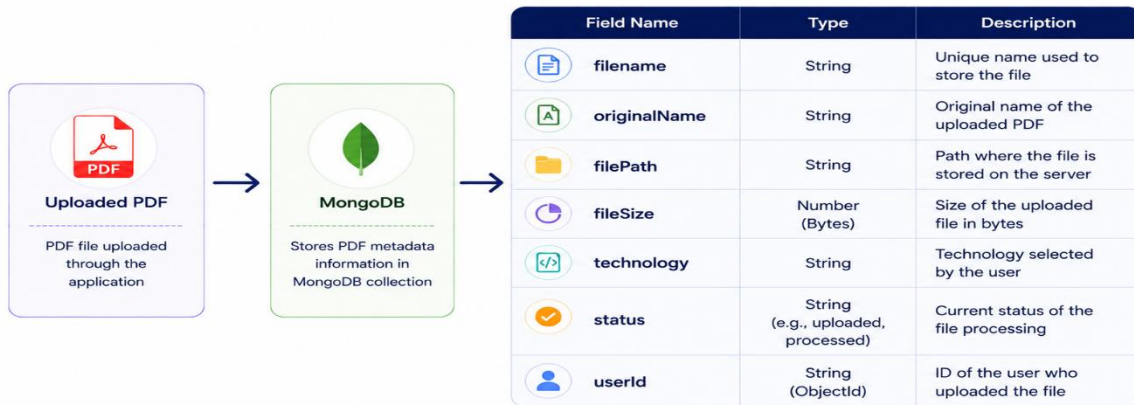


2.2 PDF File Storage

The actual PDF file is stored in the backend's upload/storage folder. MongoDB does not store the complete PDF file in the PDF metadata document. Instead, the database stores the **file path** of the uploaded PDF.



MongoDB Stores PDF Information



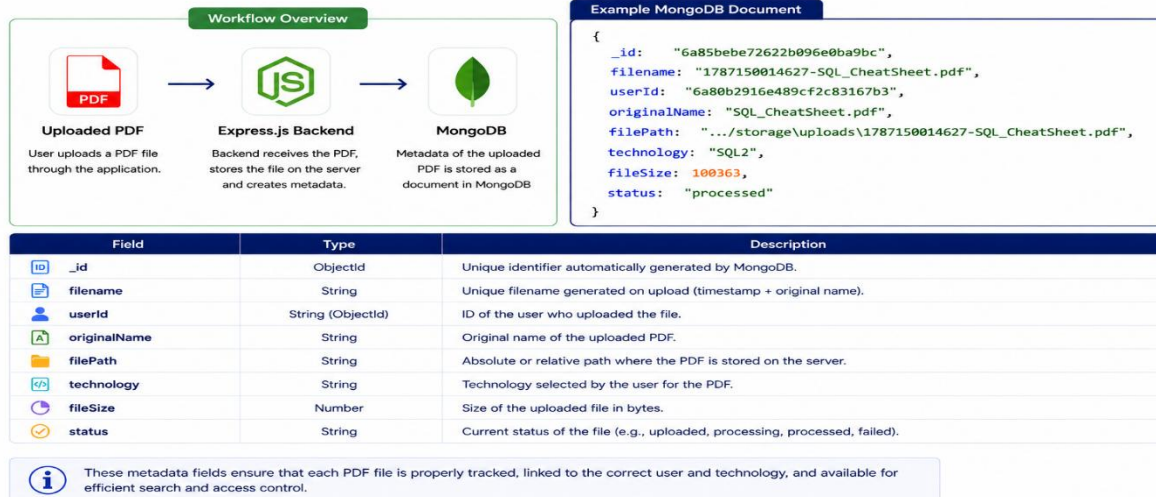
This approach keeps the actual document in file storage while MongoDB stores the information required to identify and manage that document.

2.3 MongoDB PDF Metadata

After receiving the PDF, the backend creates a PDF document in MongoDB. The PDF document contains metadata about the uploaded file.

3 MongoDB PDF Metadata

The backend stores metadata of each uploaded PDF file in MongoDB to help the system organize, retrieve and manage PDF files effectively.



The `_id` identifies the PDF document in MongoDB, while `filePath` identifies where the actual PDF file is stored.

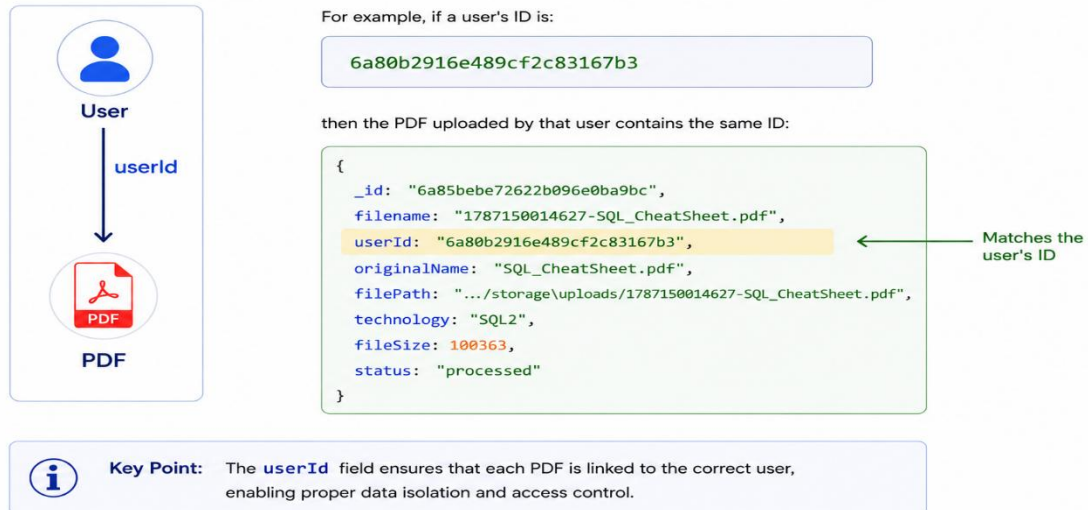
The `status` field is used to track the processing state of the document, such as uploaded, processing, processed, or failed.

2.4 User ID and PDF Mapping

Each uploaded PDF is associated with the user who uploaded it. This is done using the `userId` field in the PDF document.

4 User ID and PDF Mapping

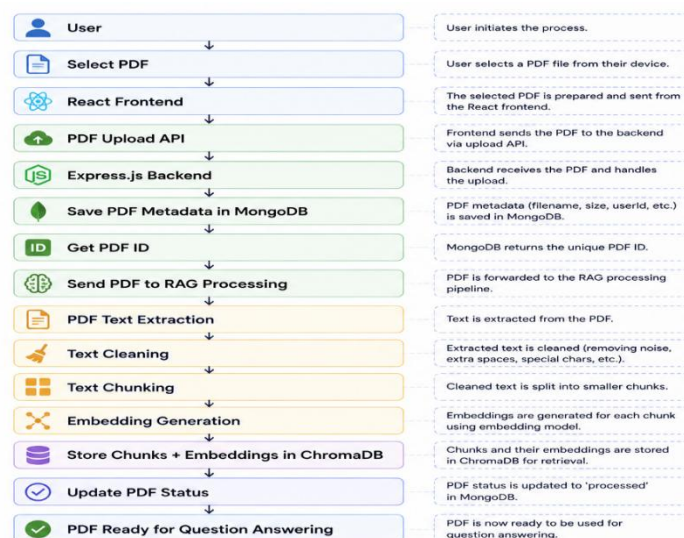
Each uploaded PDF is associated with the user who uploaded it. This is done using the `userId` field in the PDF document.



This mapping is important because the application may contain PDFs uploaded by many different users. The backend can use the `userId` to identify which PDFs belong to the current user.

2.5 PDF Upload and Processing Workflow

After the PDF metadata is saved, the PDF is passed to the RAG processing service. The PDF ID, user ID, filename, file path, and technology information are passed to the processing function.



The PDF ID and user ID are important throughout this workflow because they allow the system to maintain the relationship between the original user, the PDF, and the processed document data.

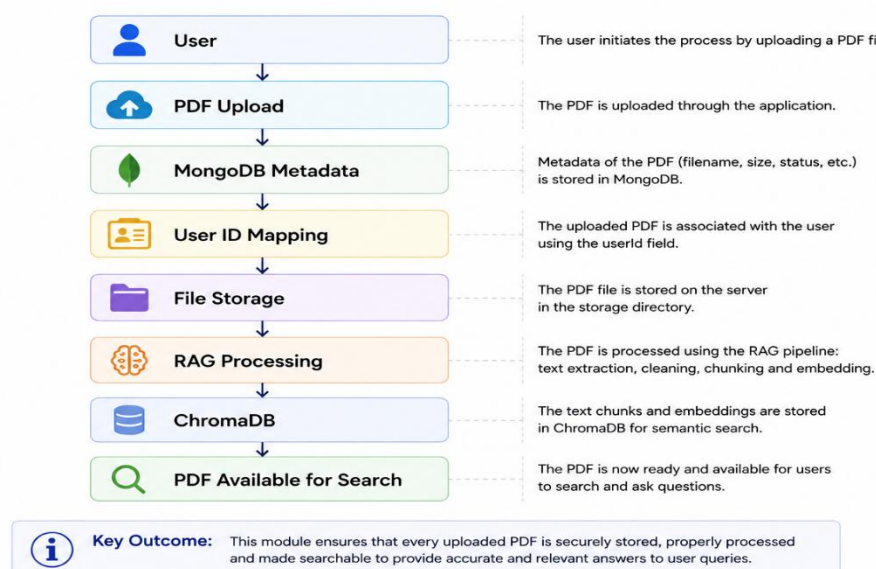
2.6 Purpose of This Module

The main purpose of this module is to create a proper connection between the user, uploaded PDF, MongoDB metadata, file storage, and RAG processing pipeline.

Once the PDF is successfully uploaded and processed, it can be displayed in the user's PDF list and its content can be used by the retrieval system when the user asks a question.

Purpose of This Module

This module handles the complete lifecycle of a PDF – from upload to making it searchable for users.



Section 3 – RAG Document Processing and Vector Storage

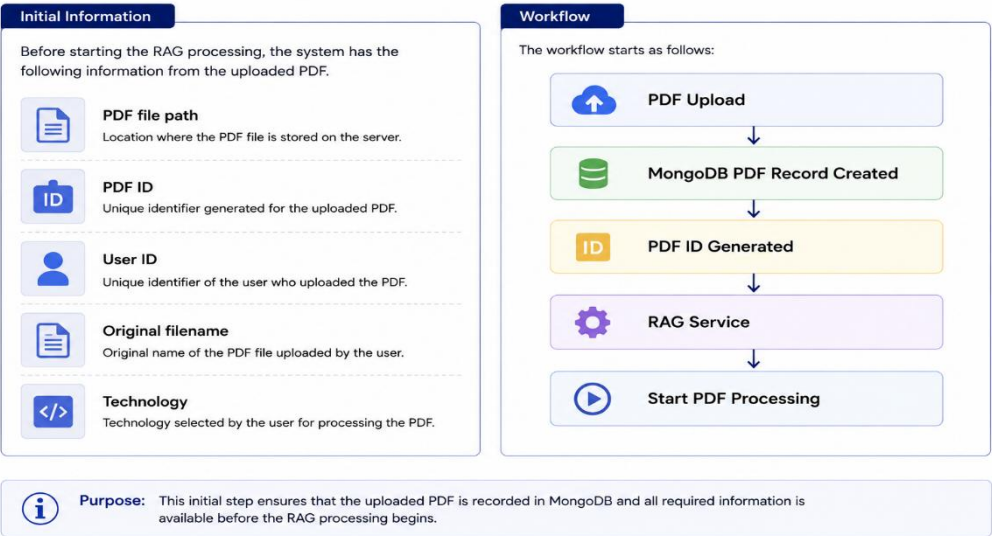
This section explains what happens after a PDF is uploaded and saved in the system. The uploaded PDF is processed through the RAG pipeline so that its content can later be searched and used to answer user questions.

The main process includes text extraction, text cleaning, chunking, embedding generation, and vector storage.

3.1 Starting the RAG Processing

After the PDF is uploaded, its metadata is first stored in MongoDB. The backend then sends the required PDF information to the RAG service.

Starting the RAG Processing



The PDF ID is important because it helps identify the PDF during retrieval. The user ID can also be used to connect the processed document with the correct user.

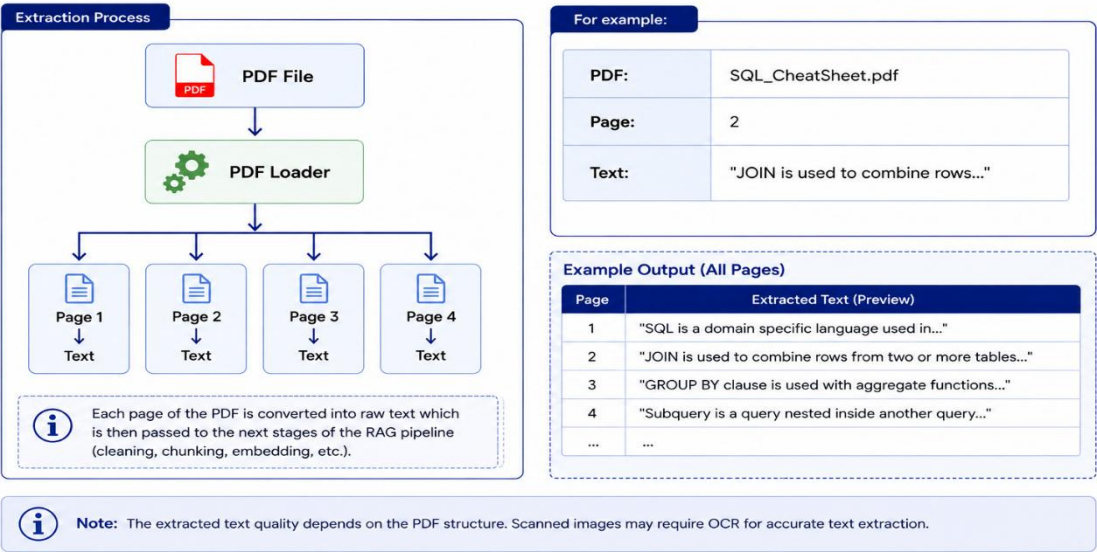
3.2 PDF Text Extraction

The first step in the RAG pipeline is to extract text from the uploaded PDF.

The PDF processing service reads the PDF page by page and extracts the text from each page.

PDF Text Extraction

The uploaded PDF file is read and text is extracted from each page. Each page's text is captured for further processing.

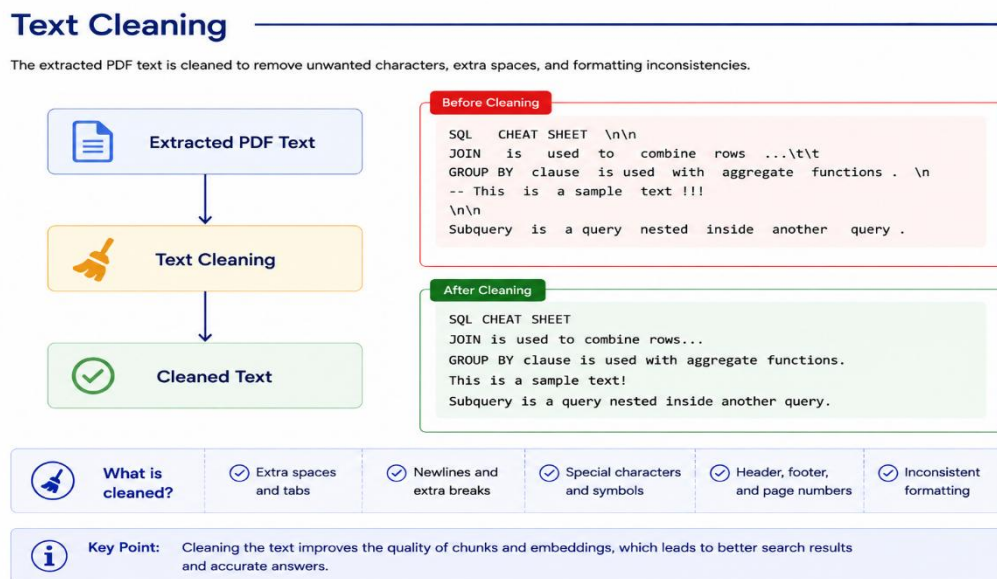


The page number is maintained with the extracted text. This is important because when an answer is generated, the system can identify the PDF and page from which the relevant information was retrieved.

3.3 Text Cleaning

The extracted PDF text may contain unnecessary spaces, empty lines, unwanted characters, or formatting issues.

Text cleaning is performed before chunking so that the document content becomes more consistent and easier to process.

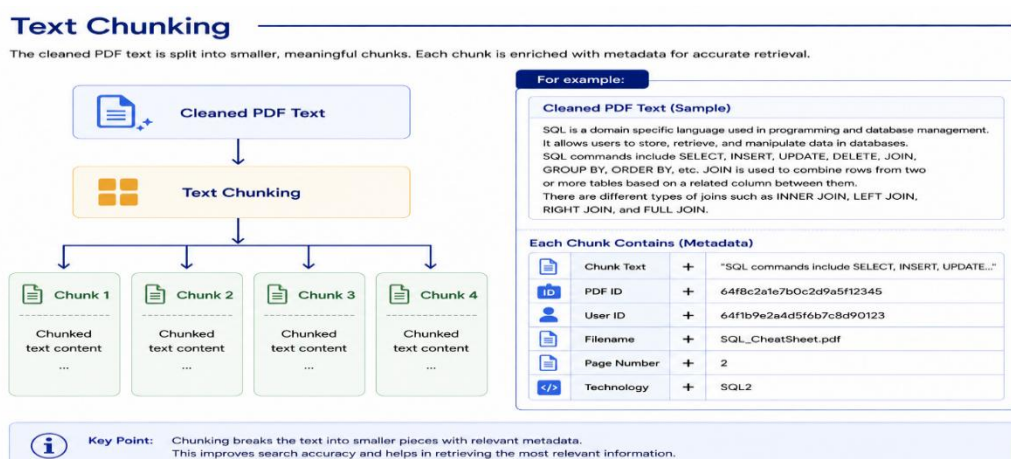


The main purpose of text cleaning is to prepare the extracted content for the next step, which is text chunking.

3.4 Text Chunking

A complete PDF can contain thousands of words. Sending the entire PDF directly to the embedding model or LLM is not efficient.

Therefore, the cleaned text is divided into smaller parts called chunks.



Each chunk contains a smaller part of the original document. The chunk also keeps important metadata related to its source.

This metadata helps the system identify where each chunk originally came from.

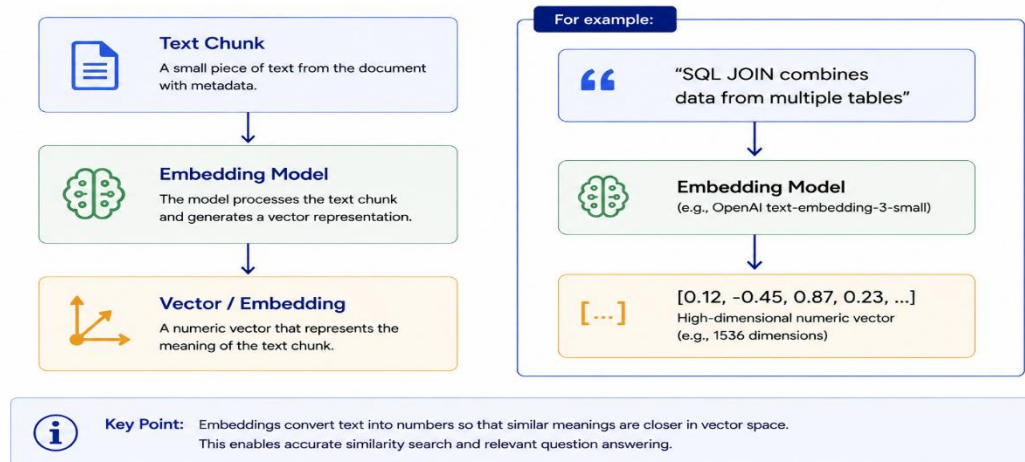
3.5 Embedding Generation

After chunking, each text chunk is converted into a numerical representation called an embedding.

The embedding model converts the meaning of the text into a vector.

Embedding Generation

Each text chunk is converted into a dense vector (embedding) using an embedding model. These embeddings capture the meaning of the text for semantic search.

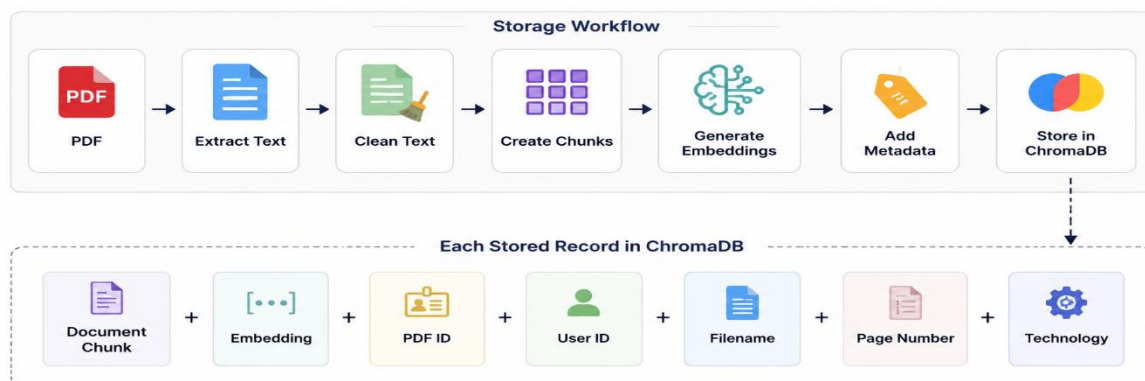


3.6 Storing Data in ChromaDB

After generating embeddings, the chunks and their metadata are stored in ChromaDB.

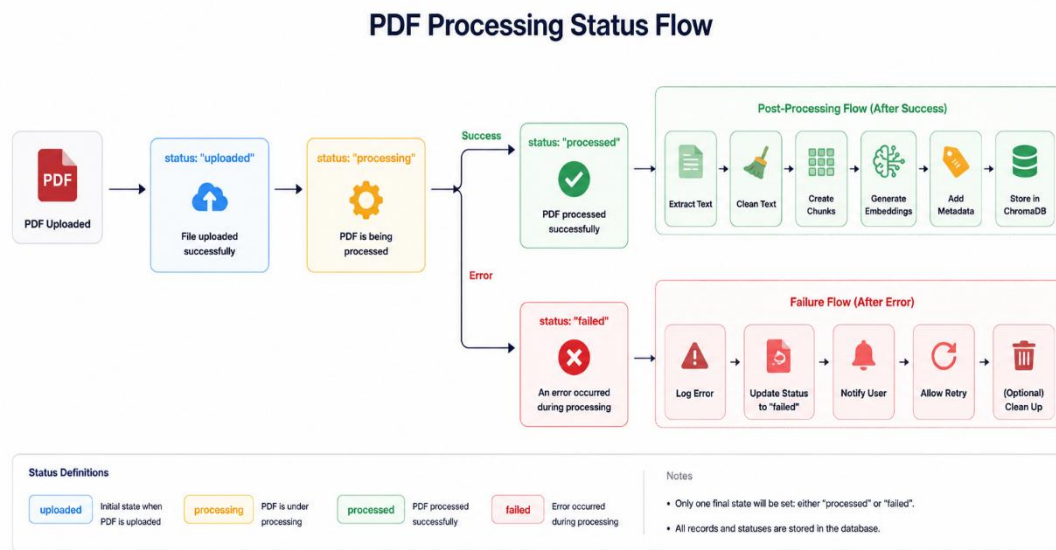
ChromaDB stores the vector representation of the document content so that it can later perform semantic search.

Storing Data in ChromaDB



3.7 PDF Processing Status

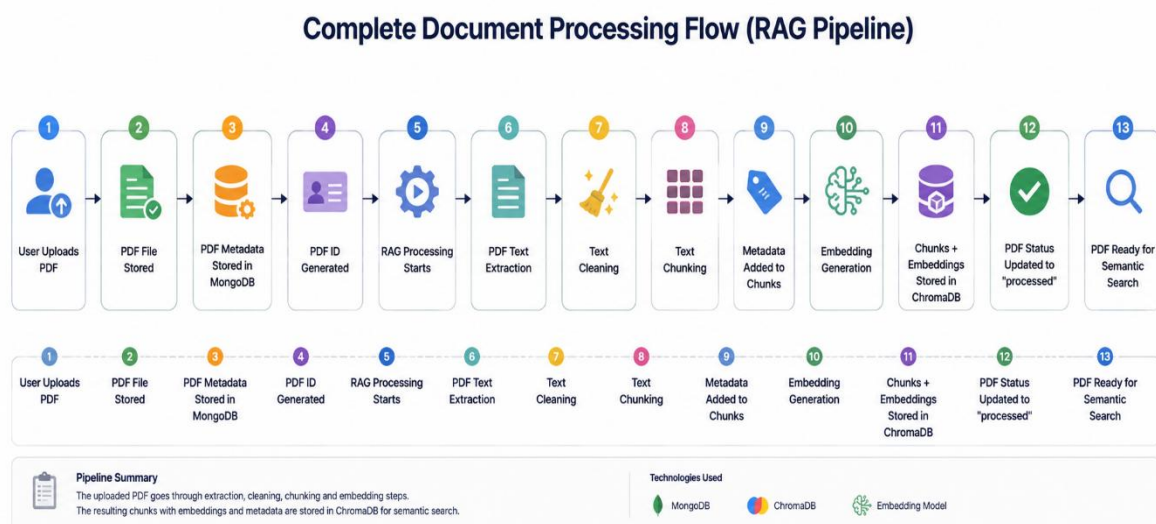
The PDF processing status is maintained in MongoDB.



This status helps the frontend and backend identify whether a PDF is ready to be used for question answering.

3.8 Complete Document Processing Flow

The complete workflow of the RAG document processing pipeline is:



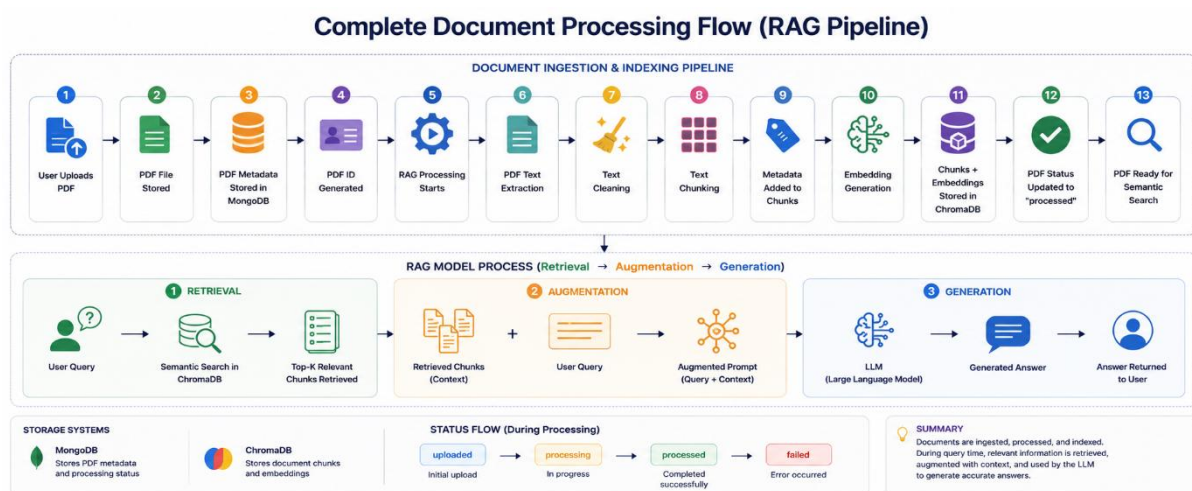
Purpose of This Section

The main purpose of this process is to convert a normal PDF into **searchable vector data**. After processing is completed, the system does not need to manually read the complete PDF every time a user asks a question.

Instead, the system can search the stored document chunks, retrieve the most relevant information, and send only the relevant content to the LLM for answer generation.

Section 4 – Question Processing, Retrieval, Augmentation and Answer Generation

This section explains what happens when a user asks a question. After the PDFs are processed and stored in ChromaDB, the user can ask questions related to the uploaded documents. The system searches the relevant content and generates an answer using the RAG pipeline.



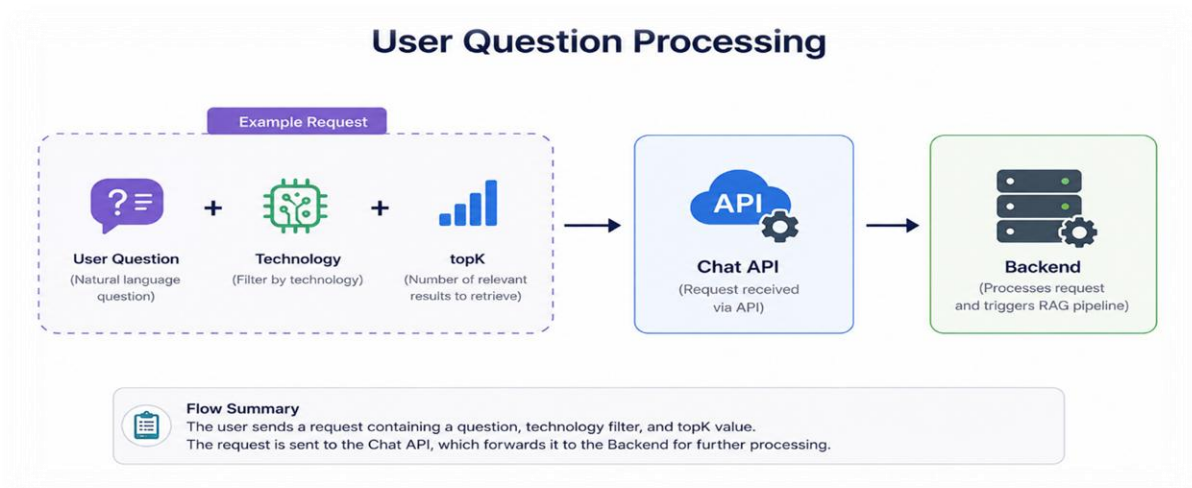
4.1 User Question Processing

The process starts when the user enters a question in the chat interface.

For example:

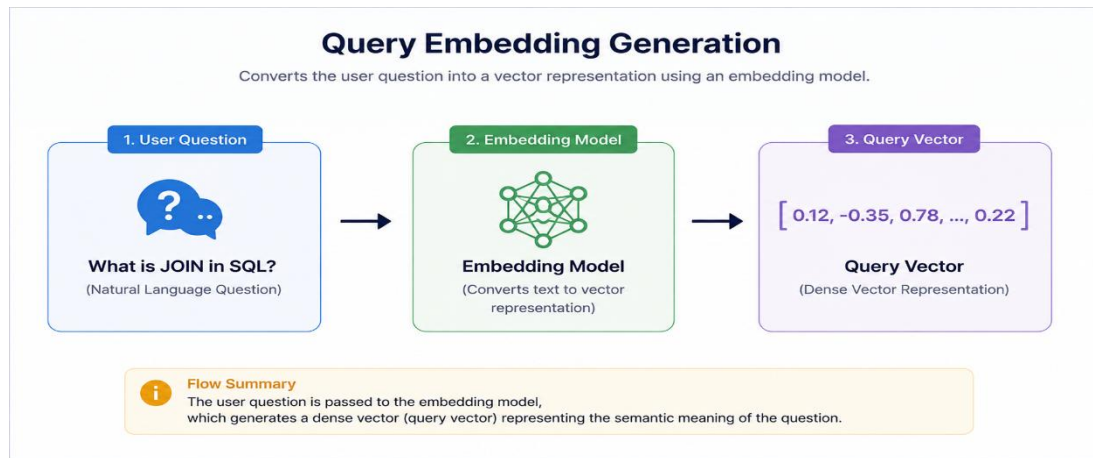
What is JOIN in SQL?

The React frontend sends the question to the backend through the Chat API. Additional information such as technology and topK can also be sent.



4.2 Query Embedding Generation

The user's question is converted into an embedding using the same embedding model used for the PDF chunks.



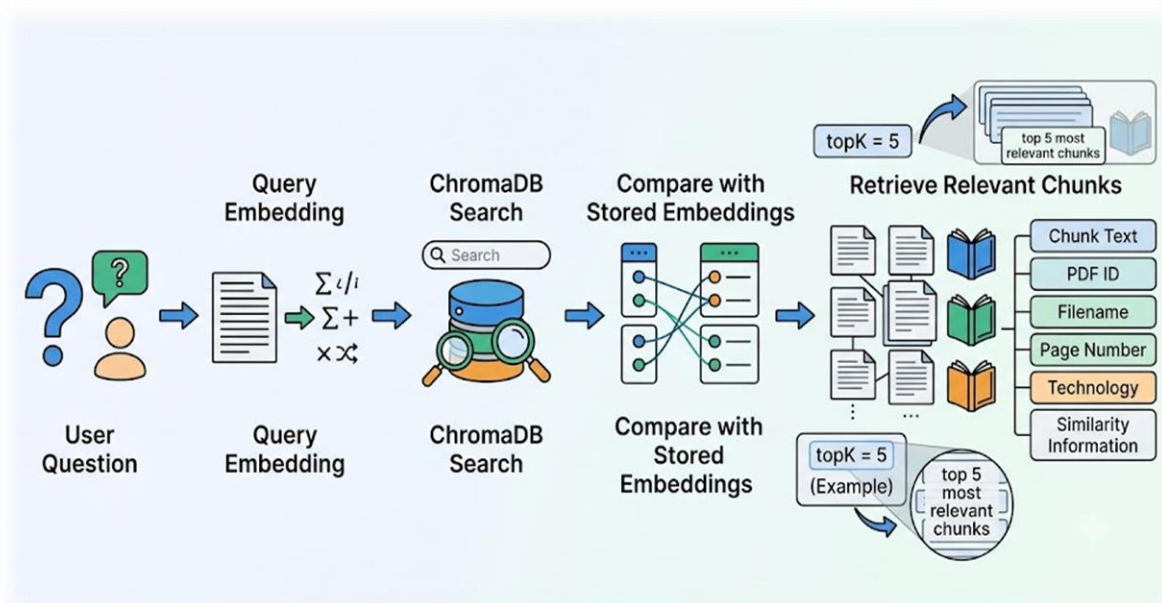
The query vector represents the meaning of the user's question. It is used to search for similar content stored in ChromaDB.

Using the same embedding model for both document chunks and user questions helps the system compare their meanings.

4.3 Retrieval

In the retrieval step, the system searches ChromaDB using the query embedding.

ChromaDB compares the user's question vector with the vectors of the stored PDF chunks and finds the most relevant chunks.

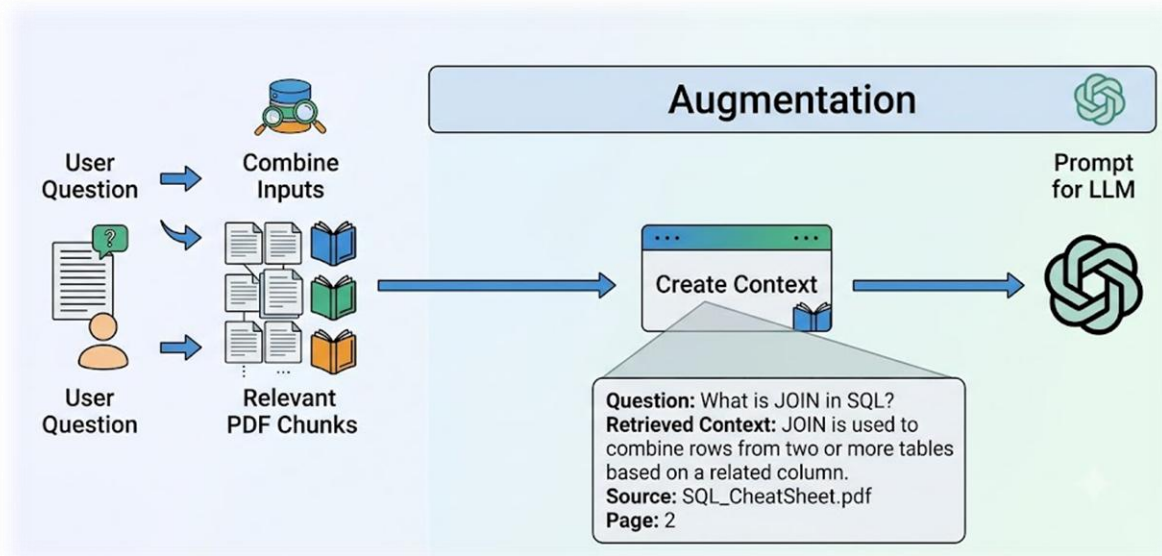


Only the most relevant content is selected instead of sending all uploaded PDFs to the LLM.

4.4 Augmentation

After retrieval, the relevant chunks are combined with the user's question.

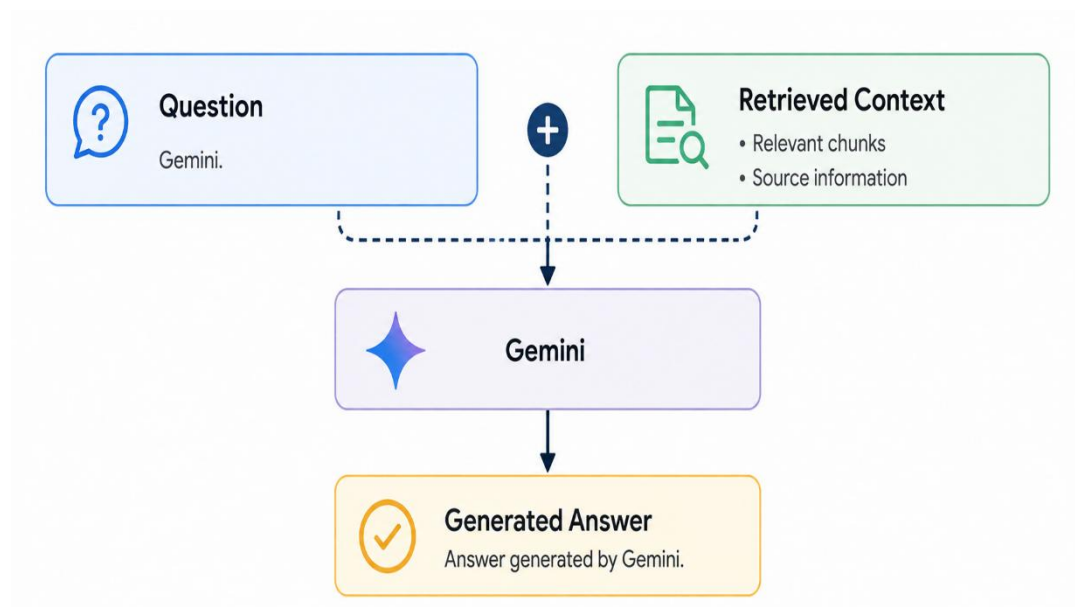
This step is called **Augmentation**.



The purpose of augmentation is to provide the LLM with the relevant information from the uploaded documents.

4.5 Generation

After the context is prepared, the user's question and retrieved PDF content are sent to Gemini.



Gemini reads the provided context and generates the final answer.

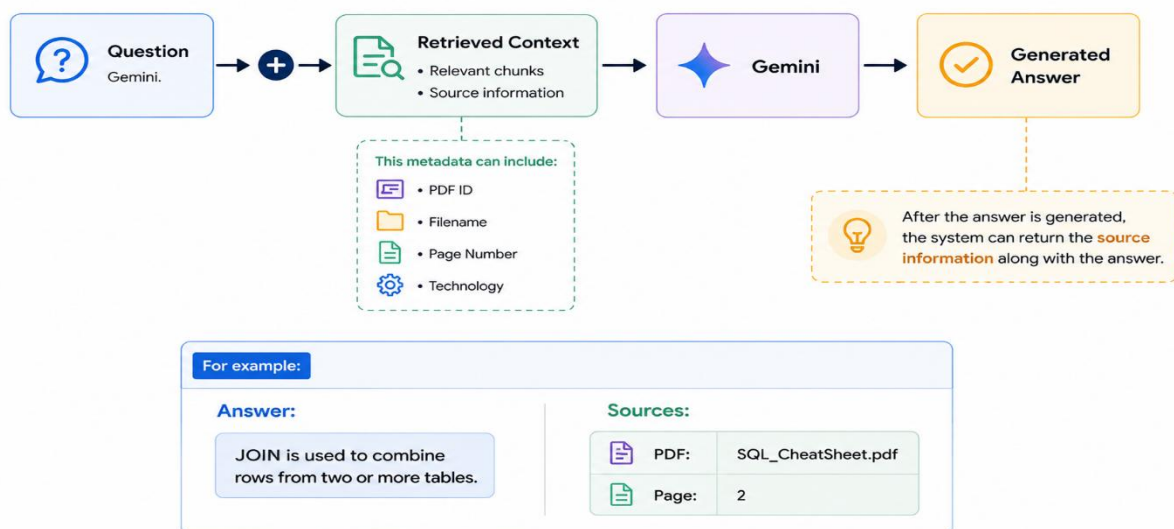
The system can also instruct the LLM to answer only from the retrieved document context. If relevant information is not available, the system can return a message such as:

The answer is not available in the uploaded documents.

This helps reduce answers based on unrelated information.

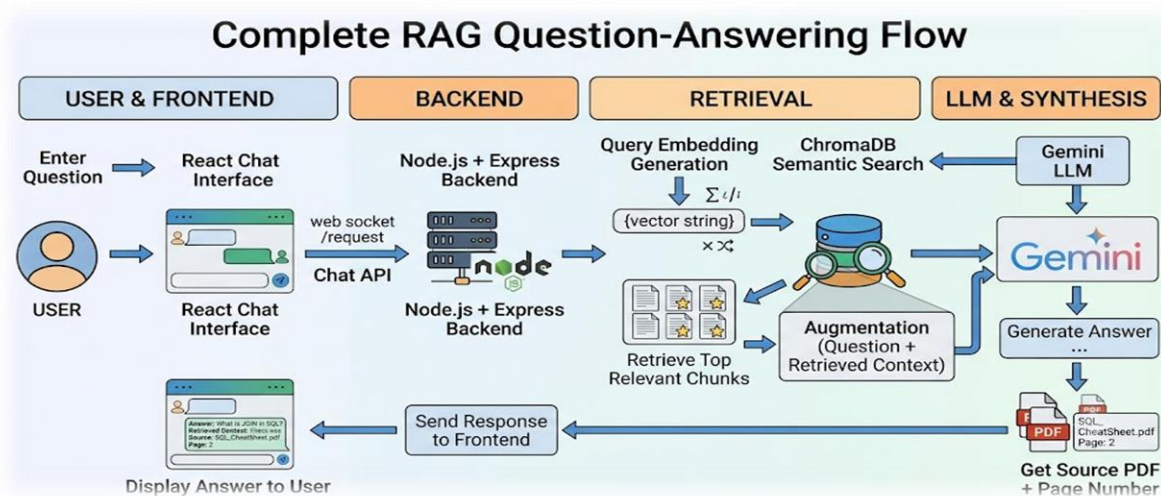
4.6 Source and Page Information

The retrieved chunks contain metadata about the original document.

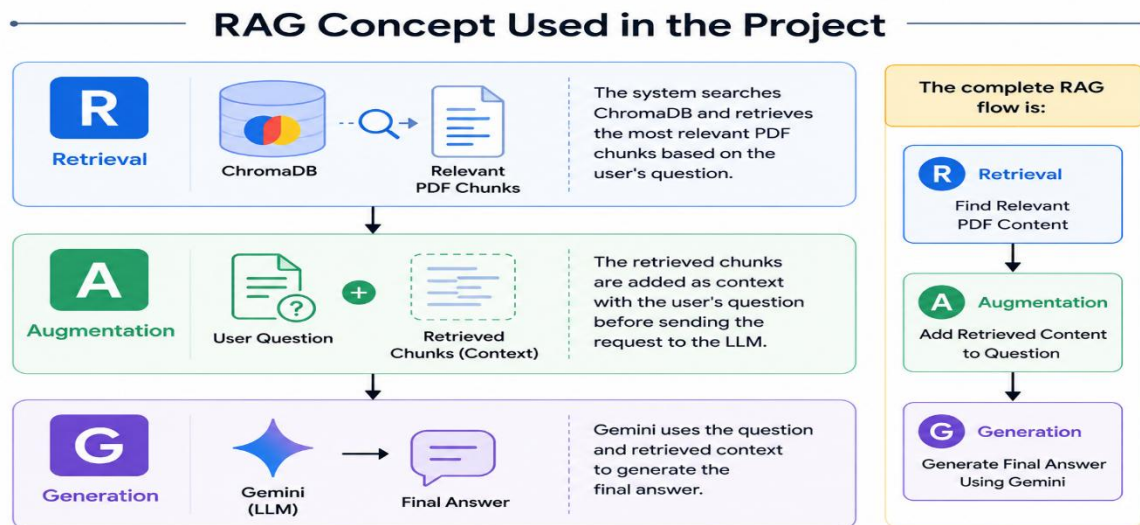


This allows the user to identify the original document and page from which the information was retrieved

4.7 Complete RAG Question-Answering Flow



4.8 RAG Concept Used in the Project



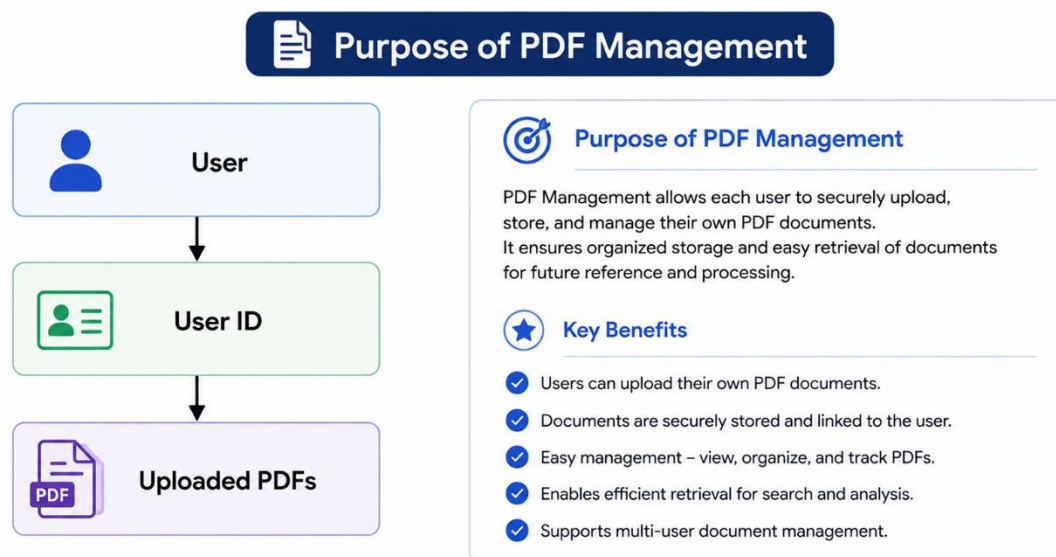
Section 5 – PDF Management and User-wise PDF List

This section explains how uploaded PDFs are managed in the application and how the system ensures that each user can access only their own PDFs.

5.1 Purpose of PDF Management

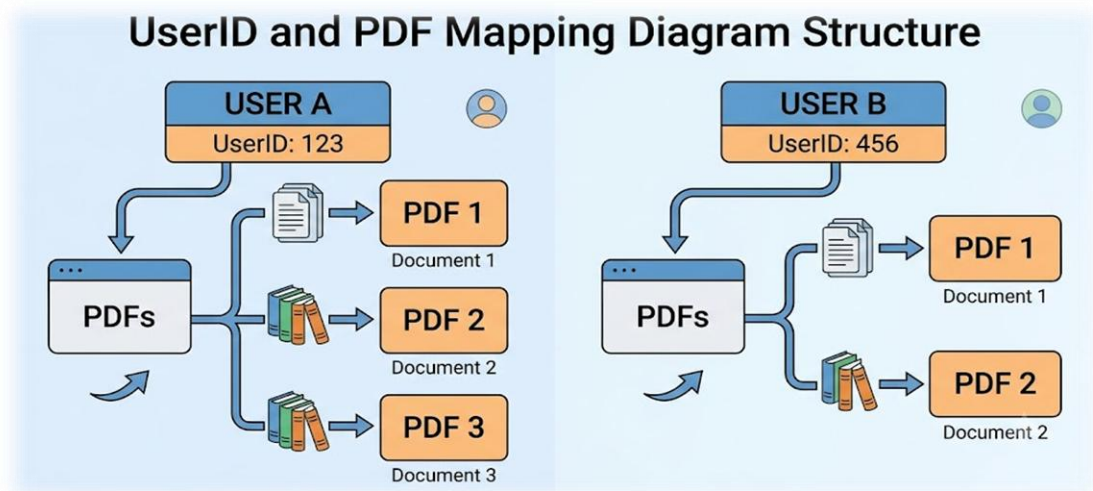
The PDF Management module is used to organize and manage the documents uploaded by users. When multiple users use the application, many PDFs can be stored in the system. Therefore, each PDF must be connected to the user who uploaded it.

For this purpose, every PDF record contains a **userId**. The **userId** creates a connection between the user and their uploaded PDF.



5.2 User ID and PDF Mapping

When a user uploads a PDF, the user's ID is stored with the PDF metadata in MongoDB.



This means the PDFs of User A and User B remain logically separated, even though all PDF metadata is stored in the same MongoDB collection.

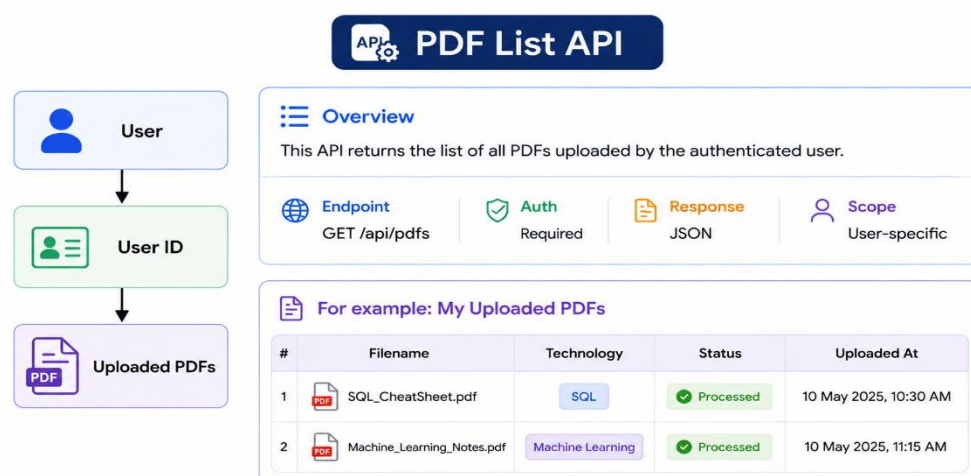
The `userId` is important because it allows the system to retrieve documents for a specific user.

5.3 PDF List API

The PDF List API is used when the user wants to view their uploaded documents.

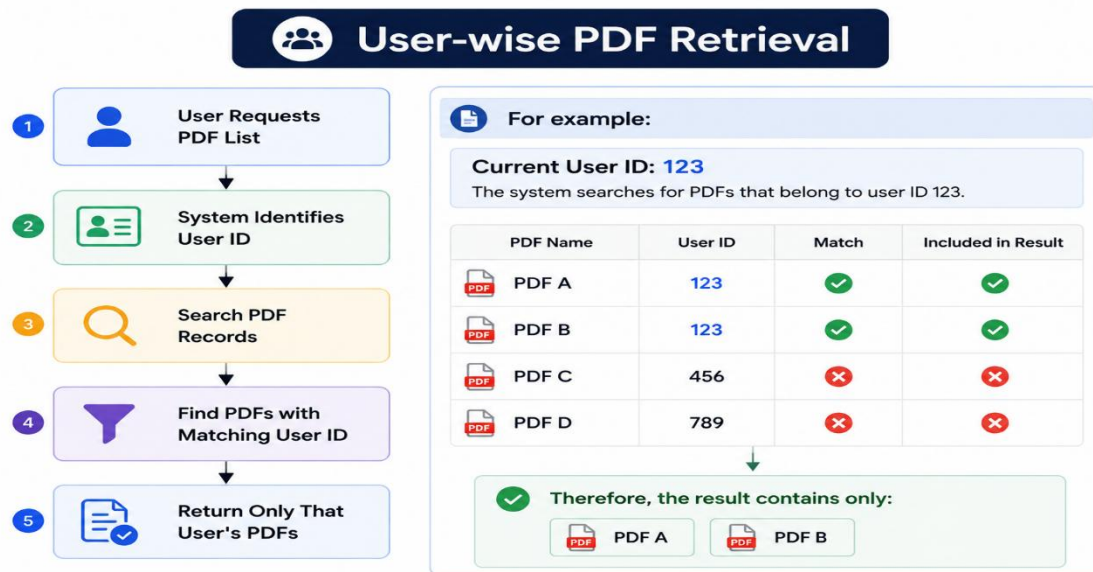
The API does not return the actual PDF file or complete PDF content. It returns the PDF metadata stored in MongoDB.

The returned information can include the PDF ID, original file name, stored file name, technology, file size, processing status, and upload date.



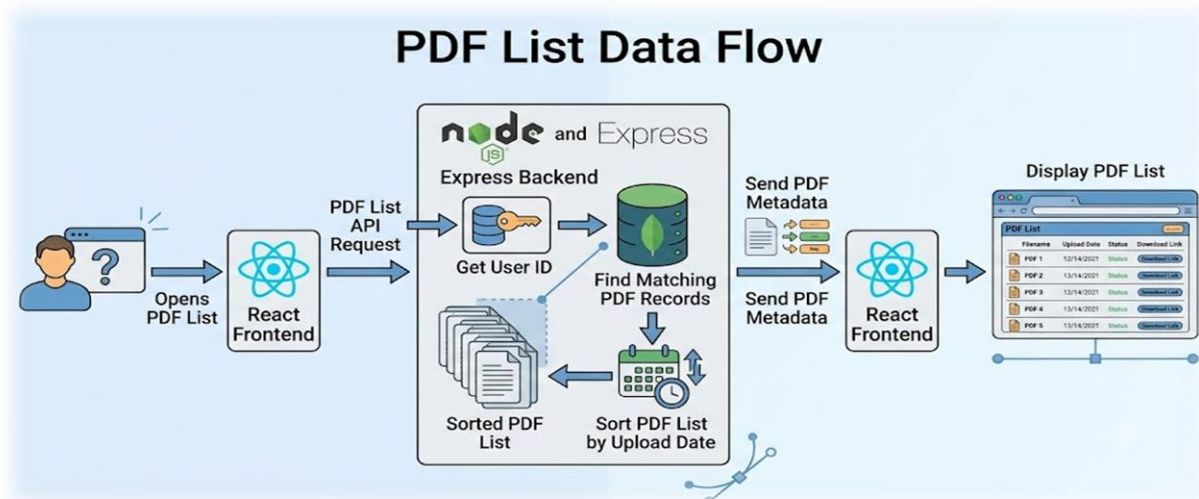
5.4 User-wise PDF Retrieval

When a user requests the PDF list, the system uses that user's ID to search the PDF records.



This ensures that one user does not see another user's PDF list.

5.5 PDF List Data Flow



The newest uploaded PDFs can be displayed first, making it easier for the user to access recently uploaded documents.

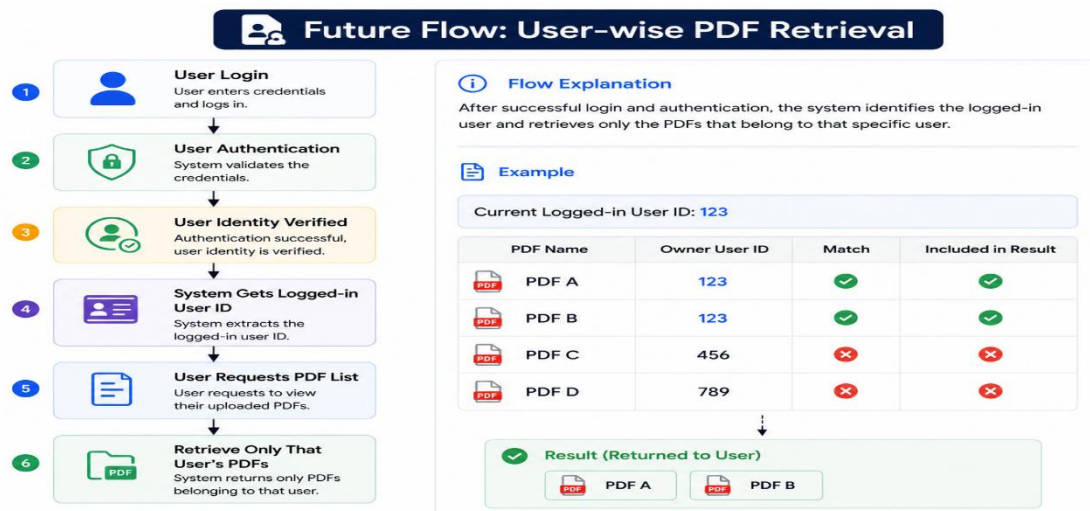
5.6 Current Development Flow

During development, authentication may not be fully connected. In that case, a temporary user ID can be used for testing the user-wise PDF list.

This allows the PDF Management and List API functionality to be developed and tested before the authentication module is completed.

After authentication is completed, the system will automatically identify the logged-in user.

The future flow will be:



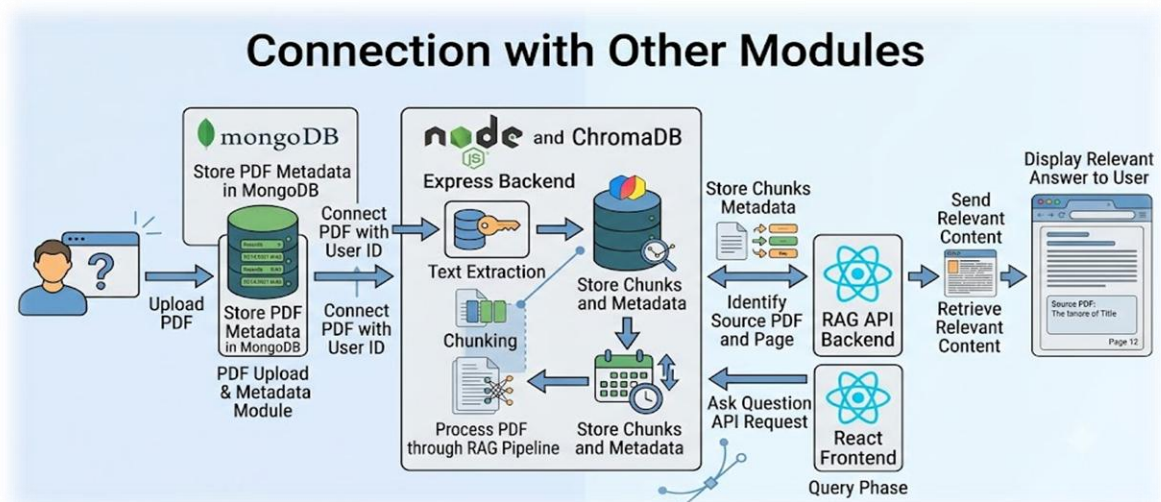
Therefore, the current PDF structure does not need to change significantly. The main improvement will be connecting the actual logged-in user's ID with the PDF operations.

5.7 Connection with Other Modules

The PDF Management module is connected with several other parts of the application.

When a PDF is uploaded, its metadata is stored in MongoDB. The same PDF is processed through the RAG pipeline, where its text is extracted, divided into chunks, converted into embeddings, and stored in ChromaDB.

The PDF ID and metadata help maintain the connection between the original PDF and its processed content.



5.8 Advantages of User-wise PDF Management

User-wise PDF management provides better organization and data separation. Each user can manage their own uploaded documents without seeing unrelated PDFs from other users.

It also makes the application easier to scale when more users start using the system. The `userId` mapping can later be used for other features such as deleting a PDF, updating PDF information, showing upload history, and managing user-specific documents.

Another advantage is that MongoDB stores only the required PDF metadata for the list, while the actual document processing data remains connected to the RAG pipeline. This keeps the PDF management and AI processing responsibilities organized.

Section 6 – Chat and Question Answering API

6.1 Purpose of the Chat API

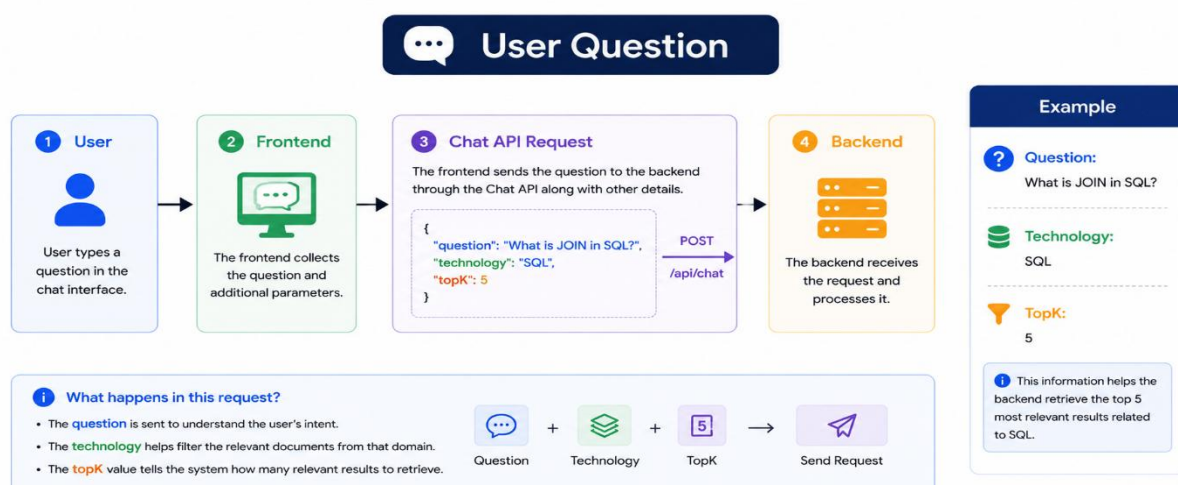
The Chat API is the main part of the application used for asking questions about uploaded PDFs. Instead of manually opening PDFs and searching for information, the user can enter a question in natural language.

The system processes the question, searches the relevant information from the user's uploaded documents, and sends the retrieved information to the LLM. The LLM then generates an answer based on the retrieved document content.

The main purpose is to provide document-based answers instead of answers based on general knowledge.

6.2 User Question

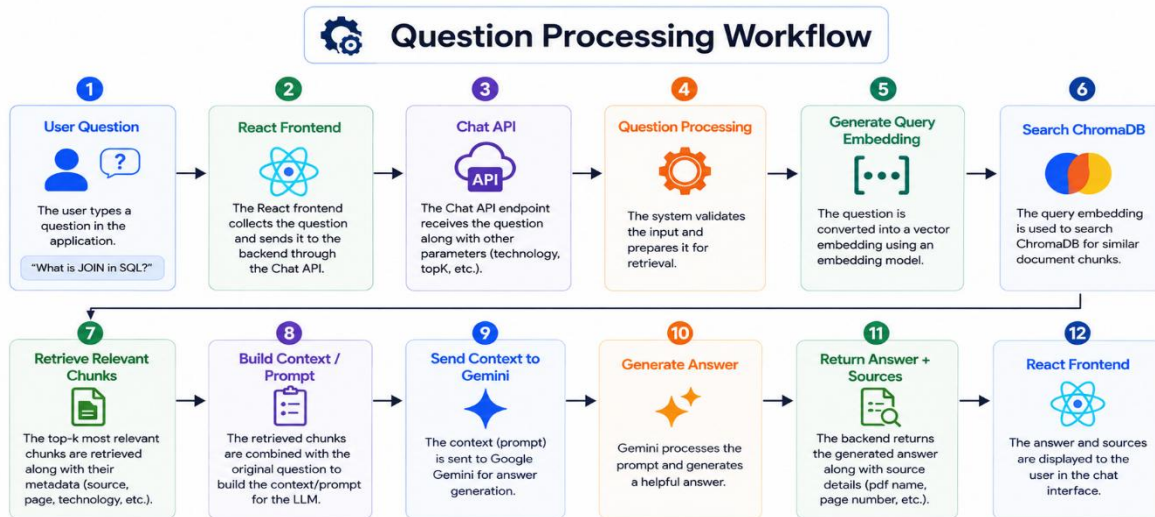
The process starts when the user enters a question in the chat interface.



Here, **TopK** indicates how many relevant chunks the retrieval system should try to find.

6.3 Question Processing Workflow

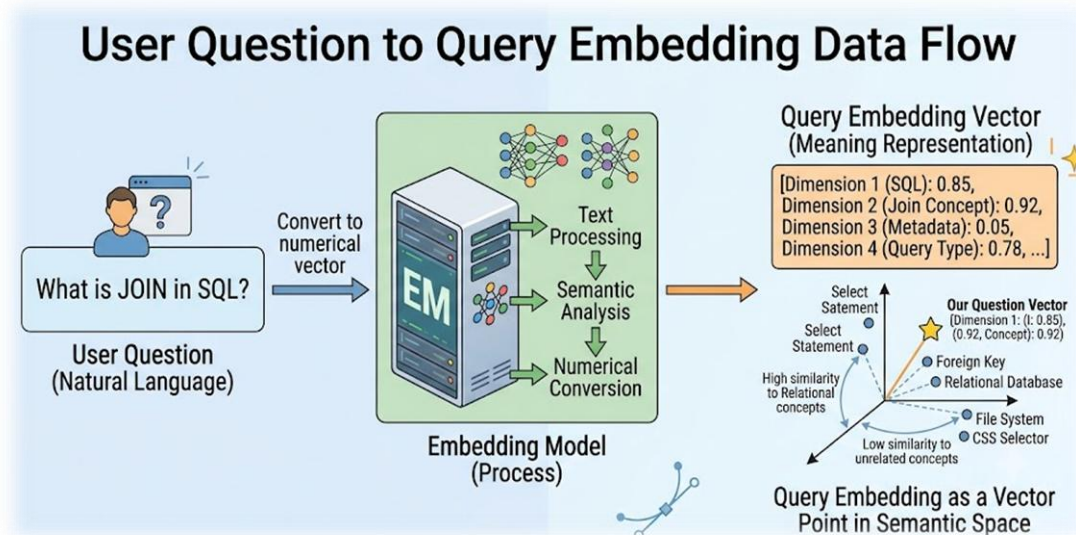
After receiving the question, the backend starts the RAG question-answering process.



The important point is that the LLM does not directly search the PDFs. The RAG pipeline first finds relevant information and then provides that information to the LLM.

6.4 Query Embedding

The user's question is converted into an embedding before searching the vector database.

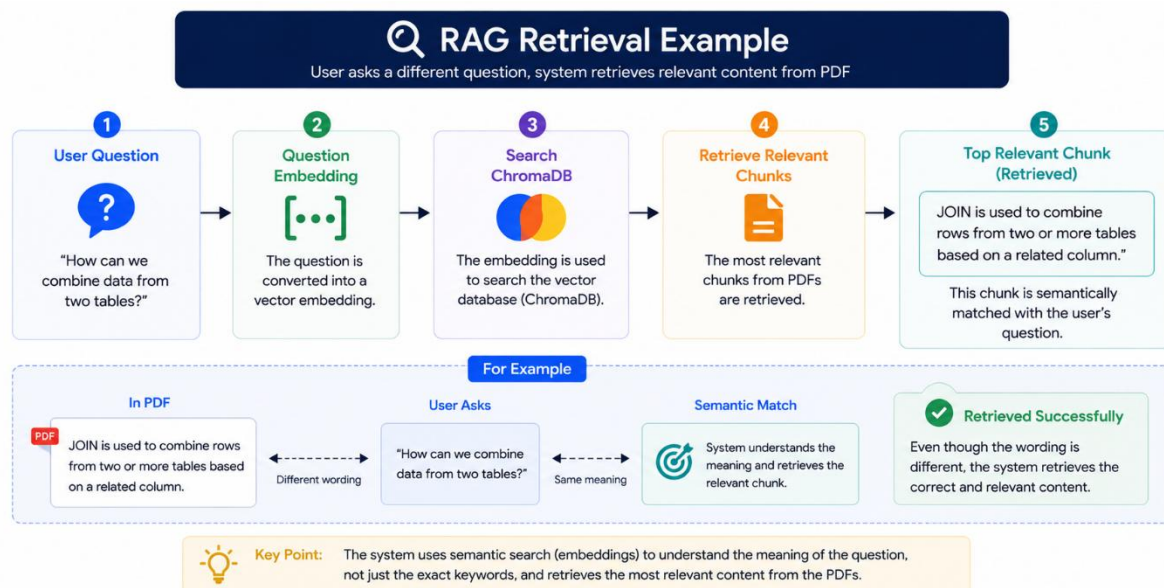


The system then compares this query vector with the vectors stored in ChromaDB.

This allows the system to find content that is semantically similar to the question, even when the exact words are different.

6.5 Retrieval

After generating the question embedding, the system searches ChromaDB for the most relevant chunks.

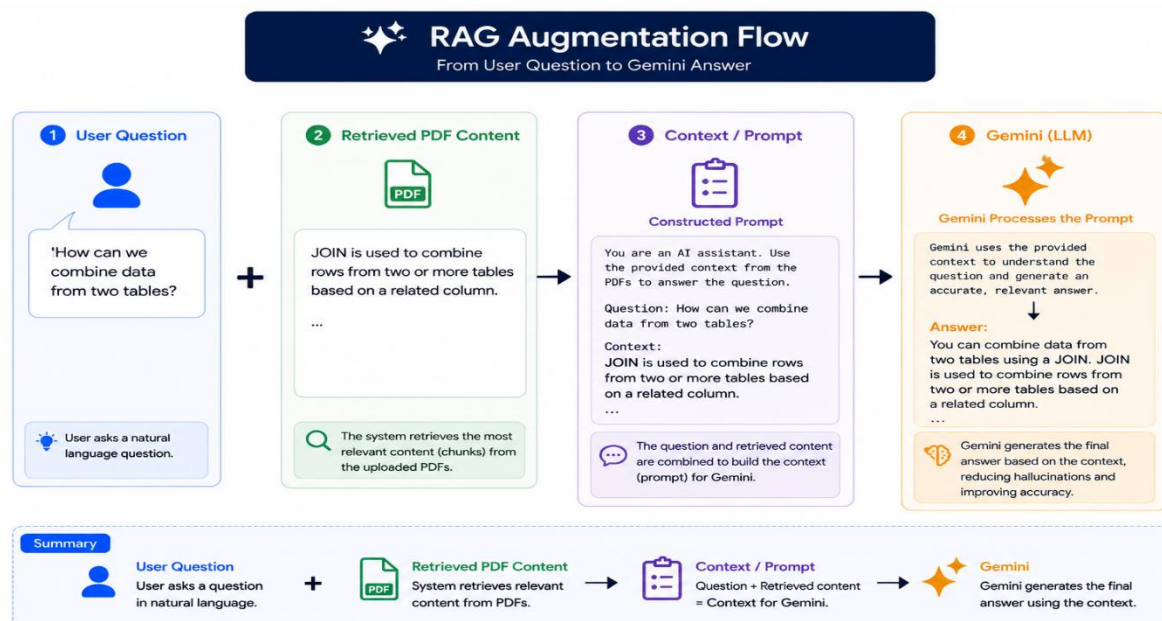


the system can still retrieve the JOIN-related chunk because the meaning of the question is similar.

The retrieved chunks become the context for the next step.

6.6 Augmentation

After retrieval, the relevant chunks are added to the prompt sent to the LLM.



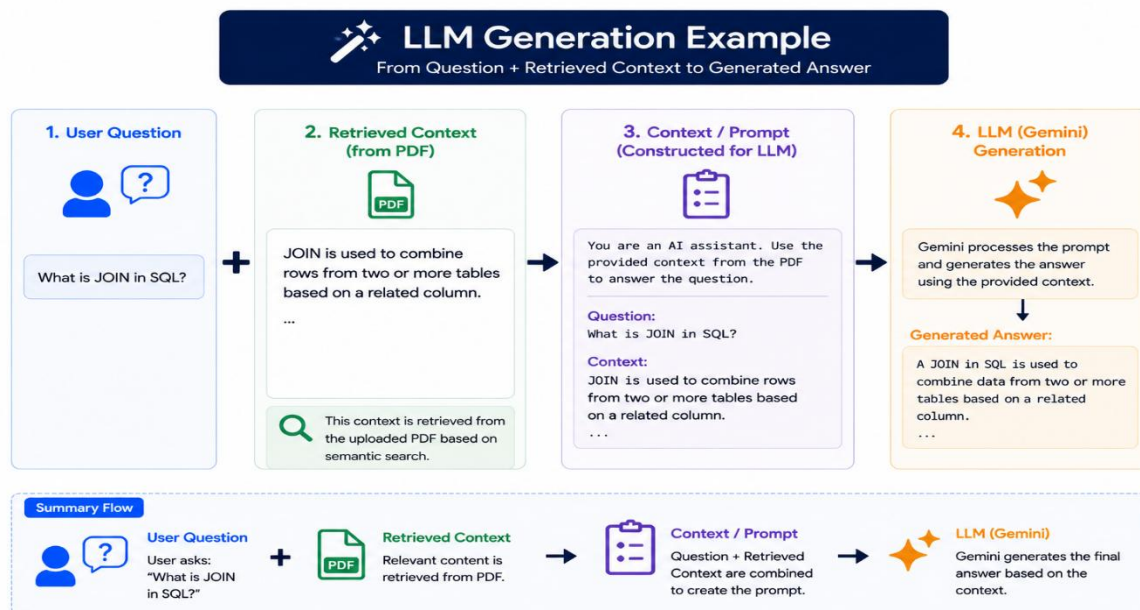
This step is called **augmentation** because the original question is enriched with information retrieved from the user's documents.

The LLM therefore receives both the question and the relevant document information required to answer it.

6.7 LLM Generation

The augmented prompt is sent to the Gemini LLM.

Gemini uses the retrieved document content to generate the final answer.



The important purpose of this process is to make the answer depend on the retrieved document content.

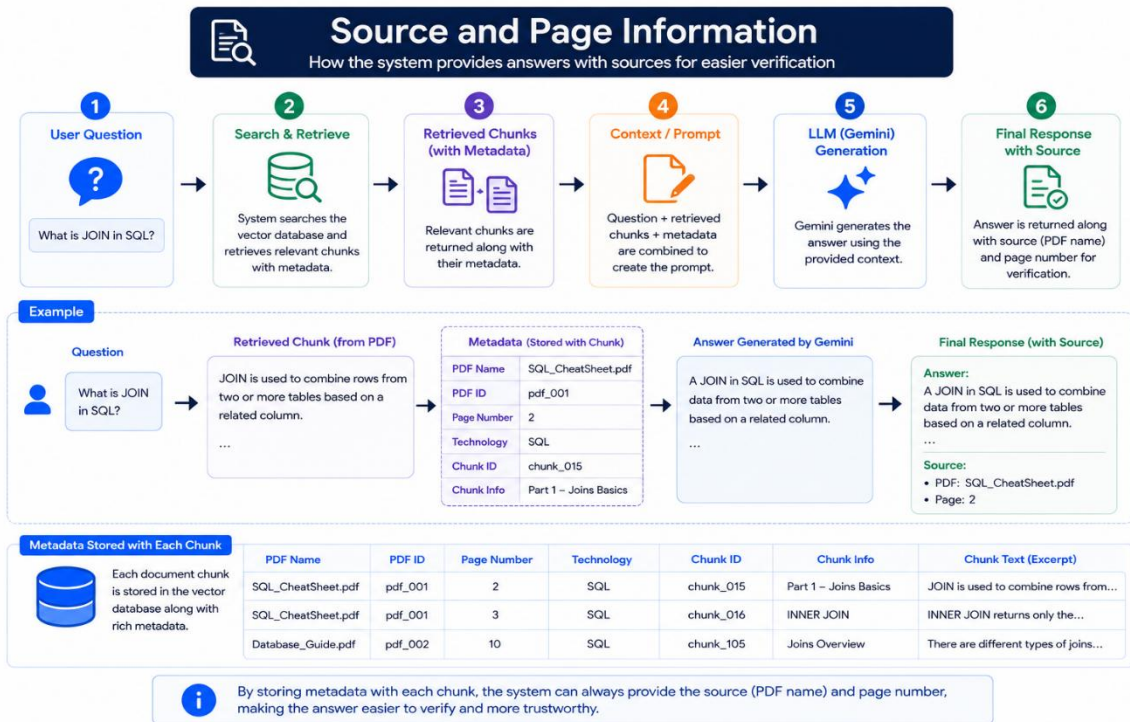
If relevant information cannot be found in the uploaded documents, the application can return a response such as:

The answer is not available in the uploaded documents.

6.8 Source and Page Information

The retrieved chunks contain metadata related to the original PDF.

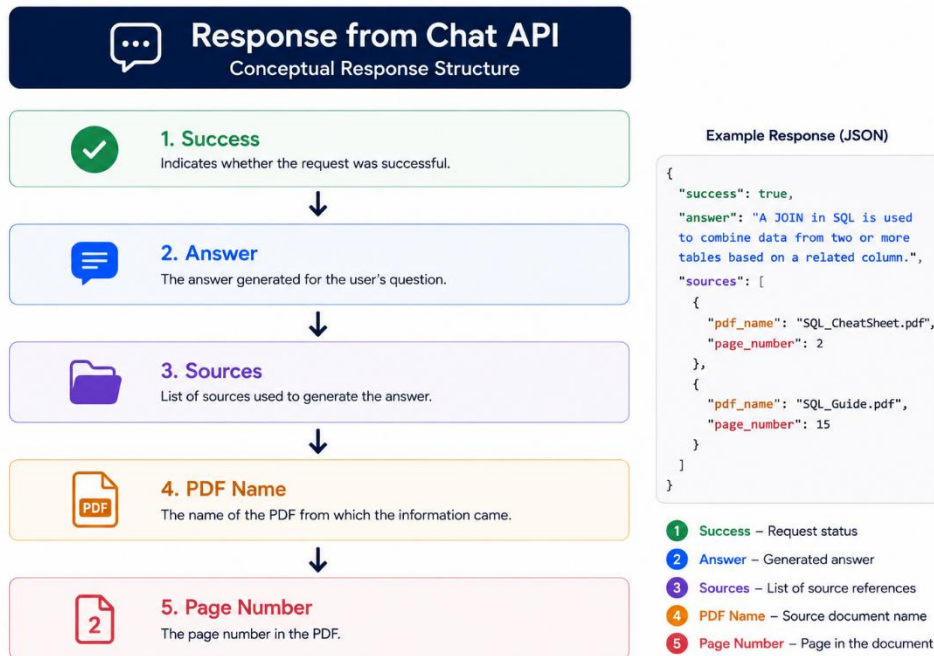
Because this information is stored along with the document chunks, the system can identify where the retrieved information came from.



6.9 Response from Chat API

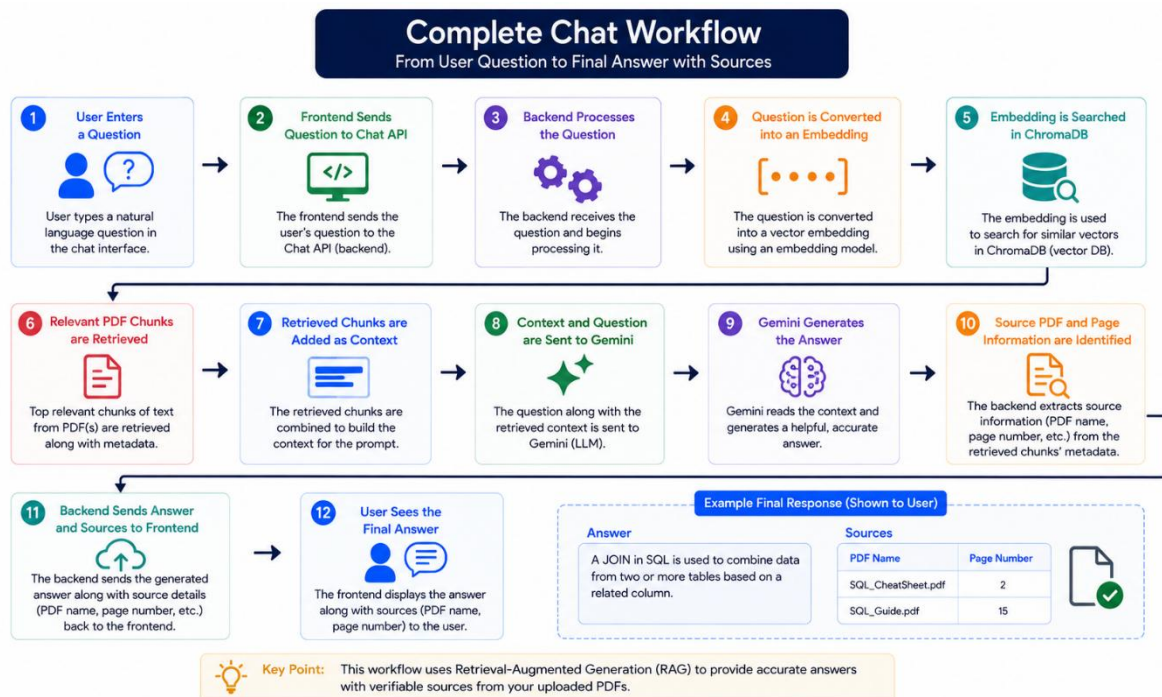
After the answer is generated, the backend sends the result back to the frontend.

The response contains the generated answer and source information.



The React frontend receives this response and displays the answer in the chat interface.

6.10 Complete Chat Workflow



6.11 Advantages of the Chat API

The Chat API allows users to search their documents using normal language instead of manually searching through multiple PDFs. Semantic retrieval helps find relevant information based on meaning, while the RAG process gives the LLM document context before generating the answer.

The source and page information also makes the generated answer easier to verify. Since the answer is generated using retrieved document content, the system can restrict responses to the knowledge available in the uploaded documents.

Section 7 – Chat History Management

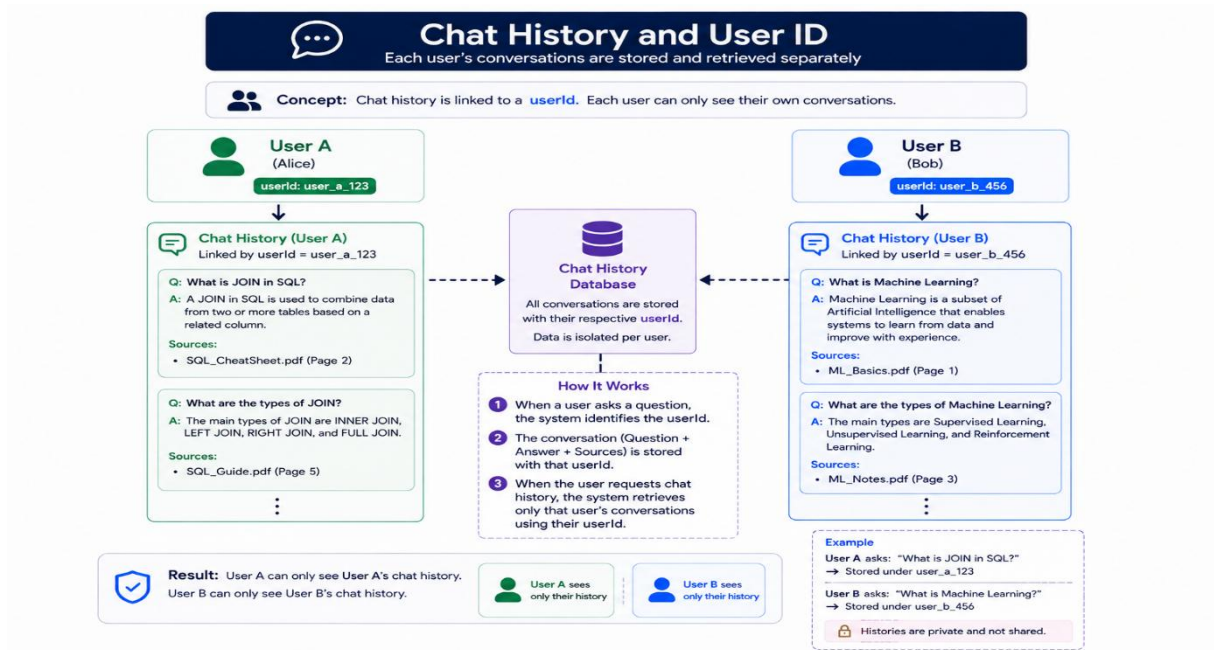
7.1 Purpose of Chat History

Chat History is used to store the conversation between the user and the AI assistant. When a user asks a question, the system generates an answer using the RAG pipeline. Both the question and the generated answer can be stored in the database.

The main purpose of chat history is to allow users to view their previous questions and answers instead of asking the same question again.

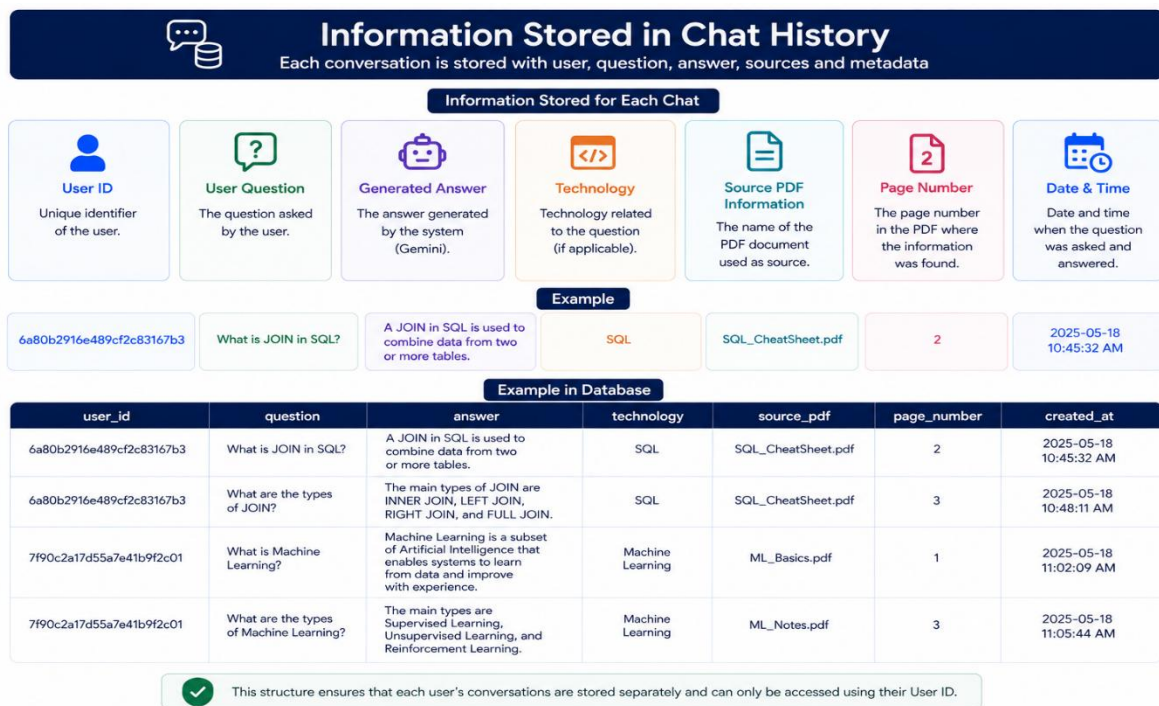
7.2 Chat History and User ID

Each chat history should be connected to a specific user through the **userId**.



7.3 Information Stored in Chat History

A chat history record can store important information related to the conversation, such as:



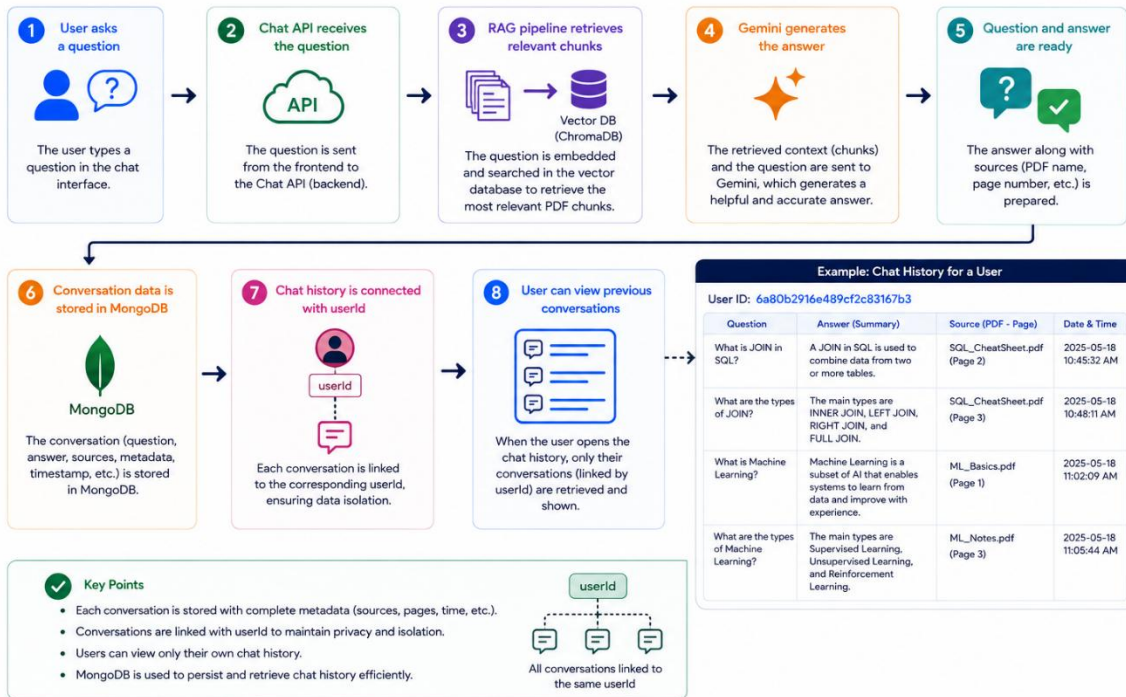
This information makes it possible to recreate the user's previous conversation.

7.4 Chat History Workflow

The chat history process starts when the user asks a question.

Chat History Workflow (End-to-End)

From User Question to Stored Conversation and History Retrieval



The answer generation and chat history storage are separate processes. First, the system generates the answer. After that, the question and answer can be saved as part of the user's conversation history.

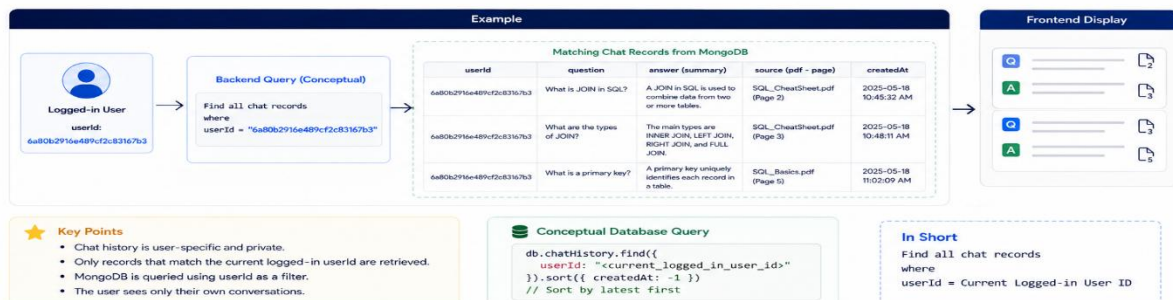
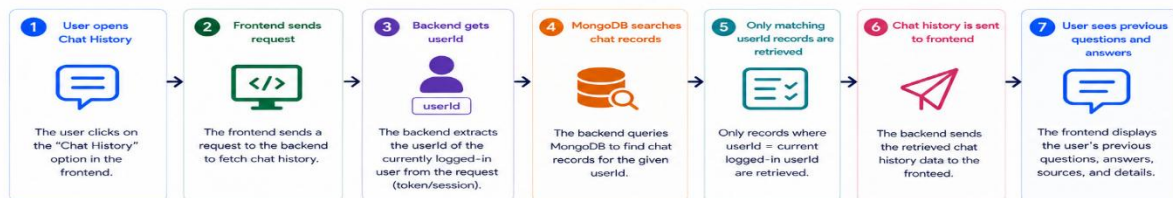
7.5 Retrieving Chat History

When the user opens the chat history section, the frontend sends a request to the backend.

The backend identifies the current user and uses the **userId** to search for that user's conversations.

Retrieving Chat History

How Chat History is Fetched and Displayed for the Logged-in User



This ensures that one user's chat history is not shown to another user.

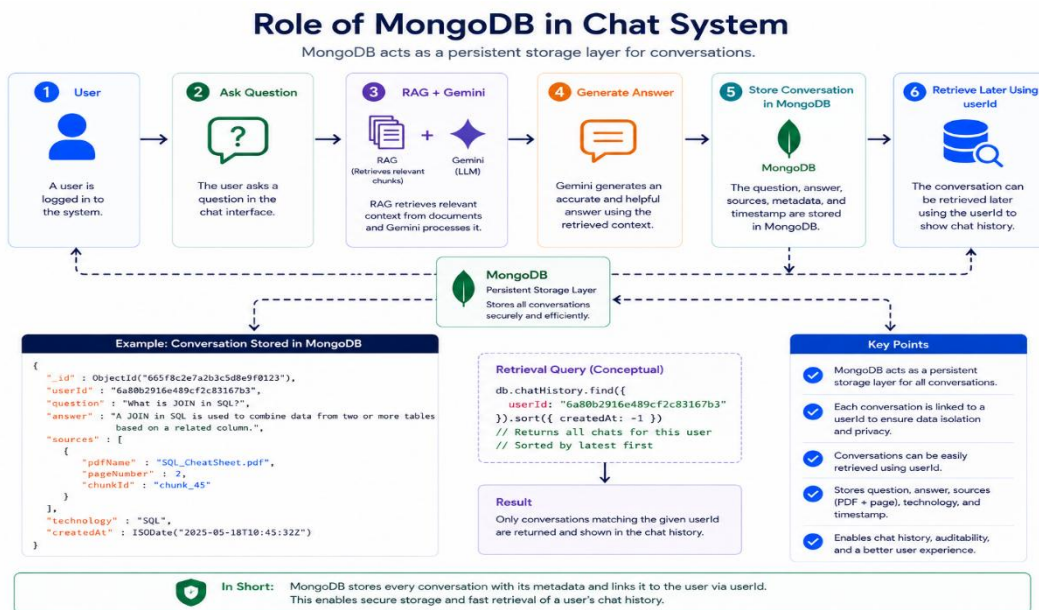
7.6 Role of MongoDB

MongoDB is used to store chat-related information permanently.

Without storing chat history, the conversation may disappear when the user refreshes the application or closes the browser.

By storing the data in MongoDB, the application can retrieve previous conversations later.

The database acts as a persistent storage layer:

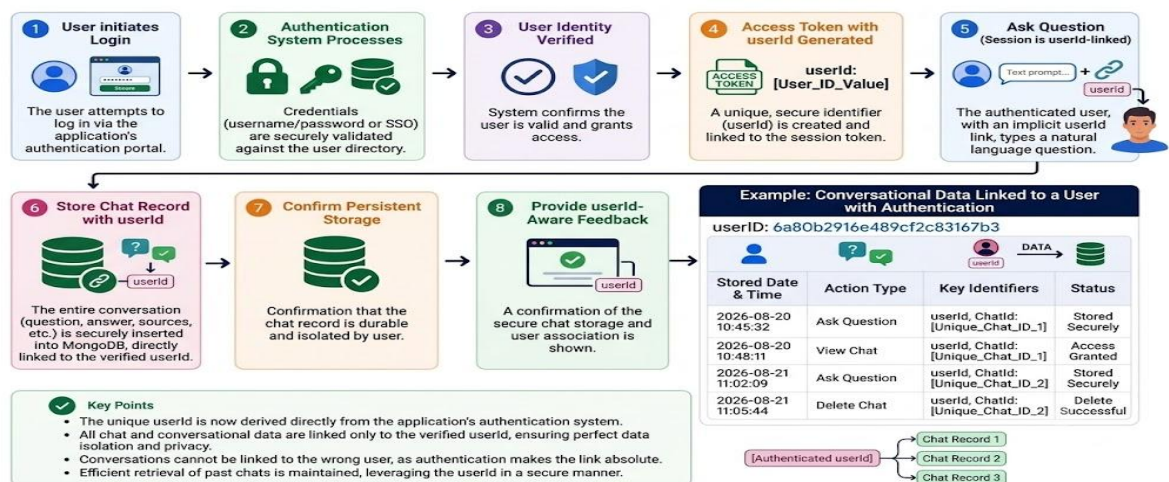


7.7 Future Integration with Authentication

Currently, during development, a user ID can be used manually for testing. After authentication is fully completed, the system should automatically identify the logged-in user.

Future Integration with Authentication Workflow (End-to-End)

From User Login to Secure, Linked Chat Storage



Because of this, the application will not need to manually enter a user ID for every request. The backend will get the user information from the authenticated request.

7.8 Advantages of Chat History

Chat history improves the user experience because users can review their previous questions and answers. It also prevents users from repeatedly searching for the same information.

Connecting each conversation with a user ID keeps the data organized and prepares the application for multi-user support. In the future, chat history can also be used to create separate conversations, search previous chats, continue an old conversation, or provide a personalized user experience.

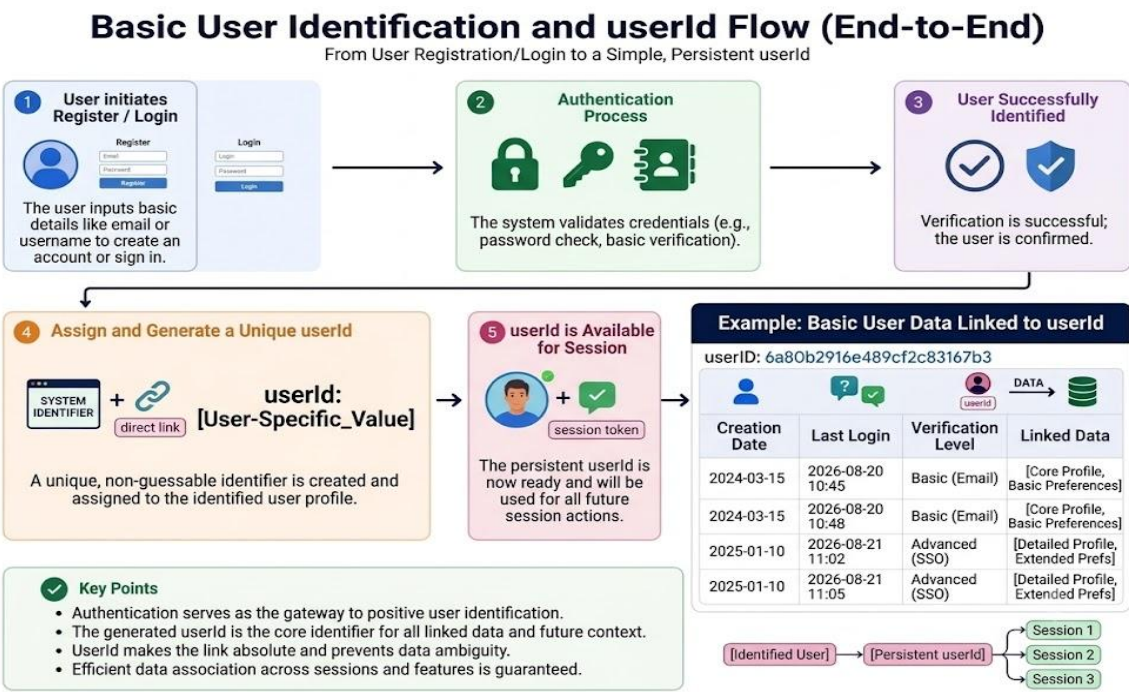
Section 8 – Complete End-to-End Project Workflow

This section explains the complete working of the **AI-Powered Research Knowledge Platform** from user registration to PDF upload, document processing, question answering, and result display.

8.1 User Authentication

The workflow starts when a user registers and logs into the application. After successful authentication, the system identifies the user using a unique **userId**.

This **userId** is important because it connects the user's data with that specific user. In the application, the **userId** can be associated with PDFs and, in the future, with chat history.



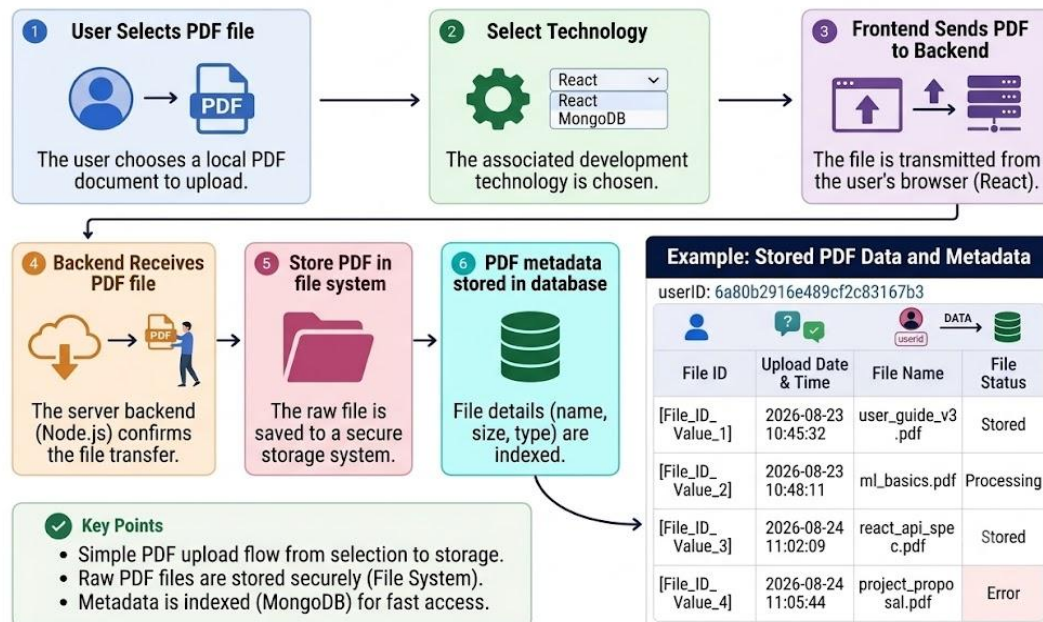
8.2 PDF Upload

After login, the user can upload a PDF document and select or provide the related technology.

When the PDF is uploaded, the backend receives the file and stores the PDF information.

The PDF metadata includes information such as the PDF filename, original name, file path, technology, file size, status, and userId.

Simple PDF Upload Workflow (End-to-End)

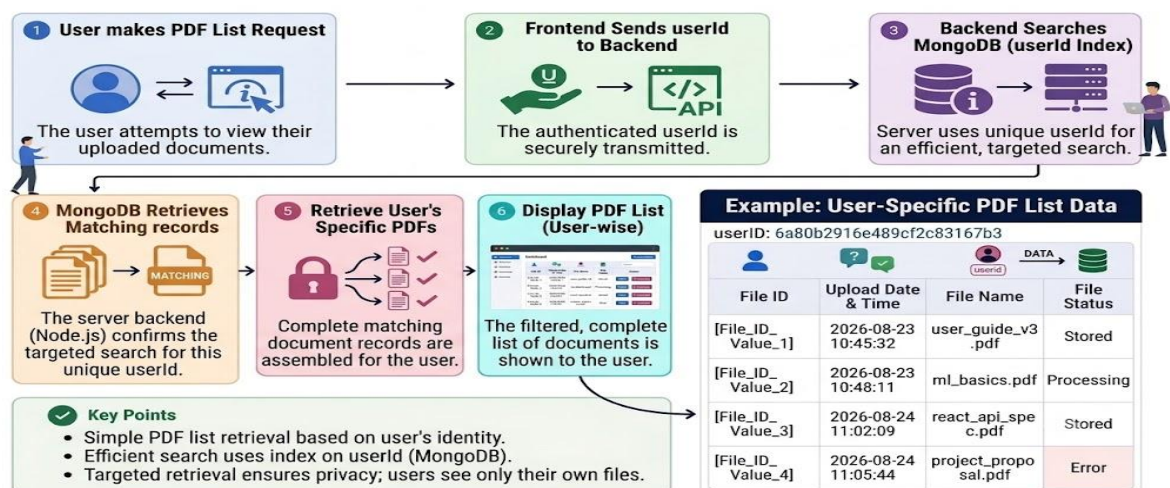


The PDF status is initially set to **uploaded**. After successful processing and indexing, the status becomes **processed**. If an error occurs during processing, the status can be changed to **failed**.

8.3 User-wise PDF Storage and Management

Each uploaded PDF can be connected with the user who uploaded it using the **userId**.

User-wise PDF Storage and Management Workflow (End-to-End)

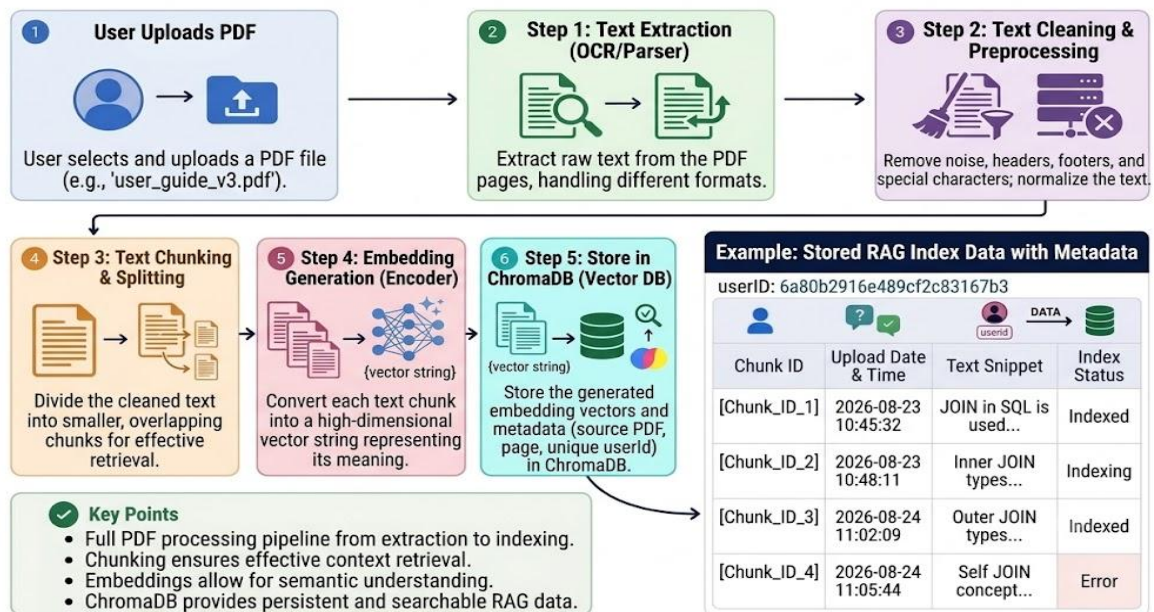


This prevents one user's PDFs from being shown to another user.

8.4 PDF Processing and RAG Indexing

After the PDF is uploaded and its metadata is stored in MongoDB, the RAG processing starts.

PDF Processing and RAG Indexing Workflow (End-to-End)



First, text is extracted from the PDF. The extracted text is cleaned to remove unnecessary spaces or unwanted formatting.

The cleaned text is then divided into smaller parts called **chunks**. Instead of processing the complete PDF at once, the system works with smaller chunks.

Each chunk is then converted into an embedding. These embeddings are stored in ChromaDB along with useful metadata such as the PDF ID, filename, page number, and other related information.

After successful indexing, the PDF status is updated to **processed**.

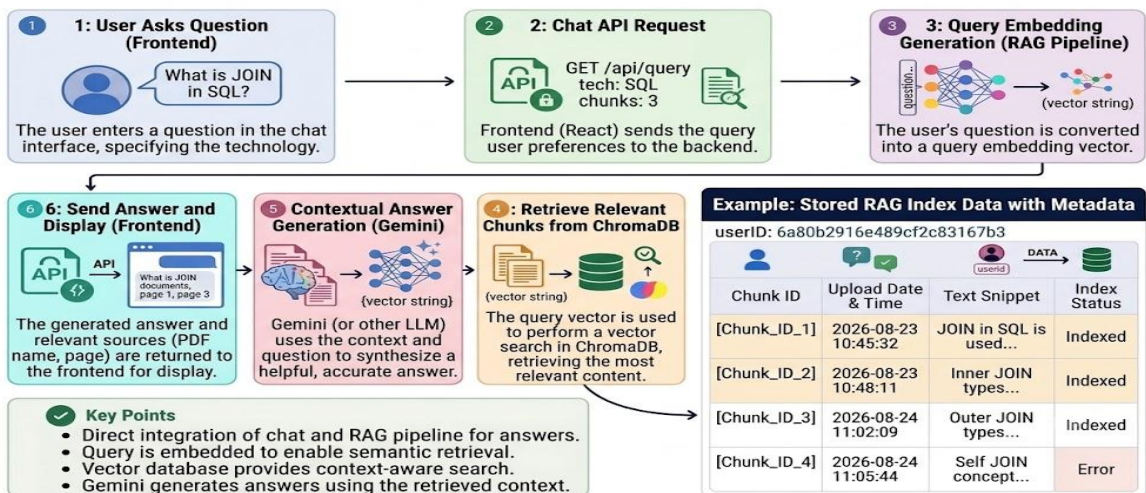
8.5 User Asks a Question

After uploading and processing PDFs, the user can ask a question through the chat interface.

The frontend sends the question to the backend through the Chat API.

The request can also include information such as the selected technology and the number of relevant chunks to retrieve.

User Asks a Question: End-to-End Query and Answer Workflow

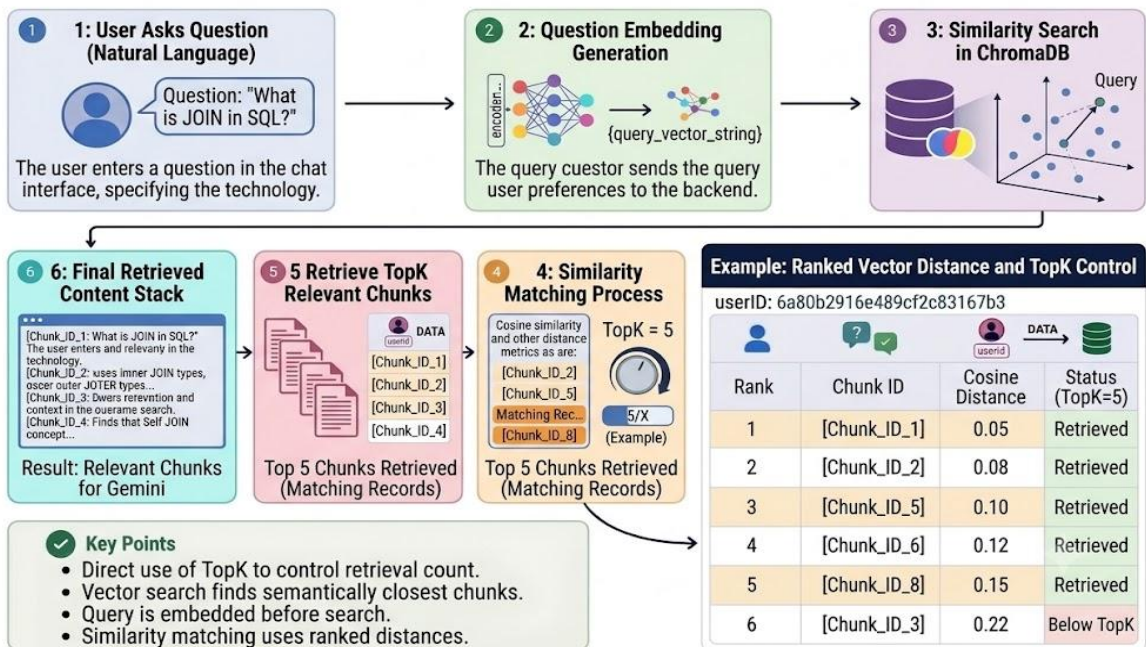


8.6 Retrieval of Relevant Information

The user's question is converted into an embedding. This embedding represents the meaning of the question.

The system searches ChromaDB and compares the question embedding with the embeddings of the stored PDF chunks.

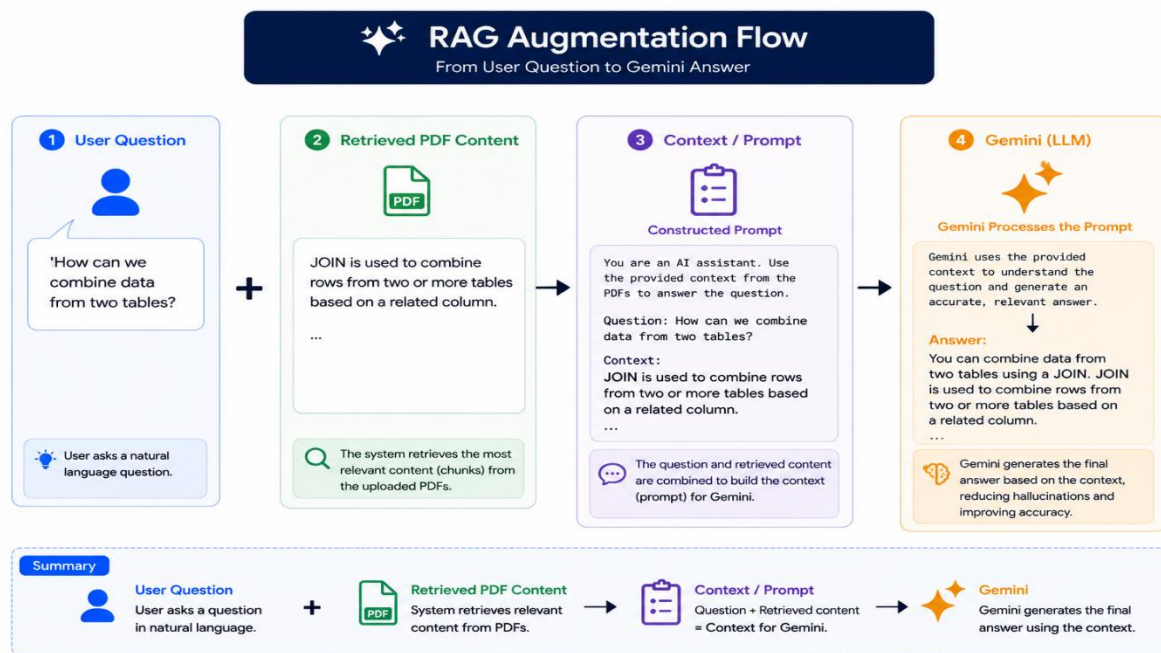
Retrieval of Relevant Information: In-Depth Vector Search Workflow



8.7 Augmentation

After retrieval, the relevant chunks are combined with the user's question.

The retrieved PDF content becomes the context for the LLM.



This step is called **Augmentation** because additional relevant information is added to the original user question before sending it to the LLM.

8.8 Answer Generation

The augmented context and user question are sent to the Gemini LLM.

Gemini uses the retrieved document content to generate the answer.



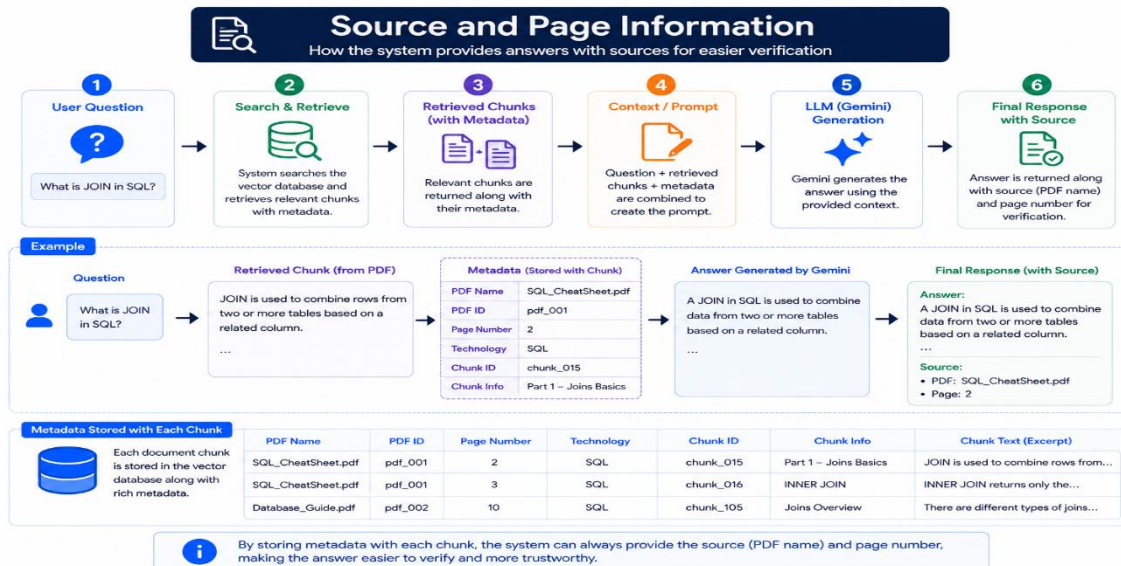
The system is designed to generate answers based on the information retrieved from the uploaded documents.

If relevant information is not found, the system can return a message such as:

The answer is not available in the uploaded documents.

8.9 Source and Page Information

The chunks stored in ChromaDB contain metadata about their original PDF.

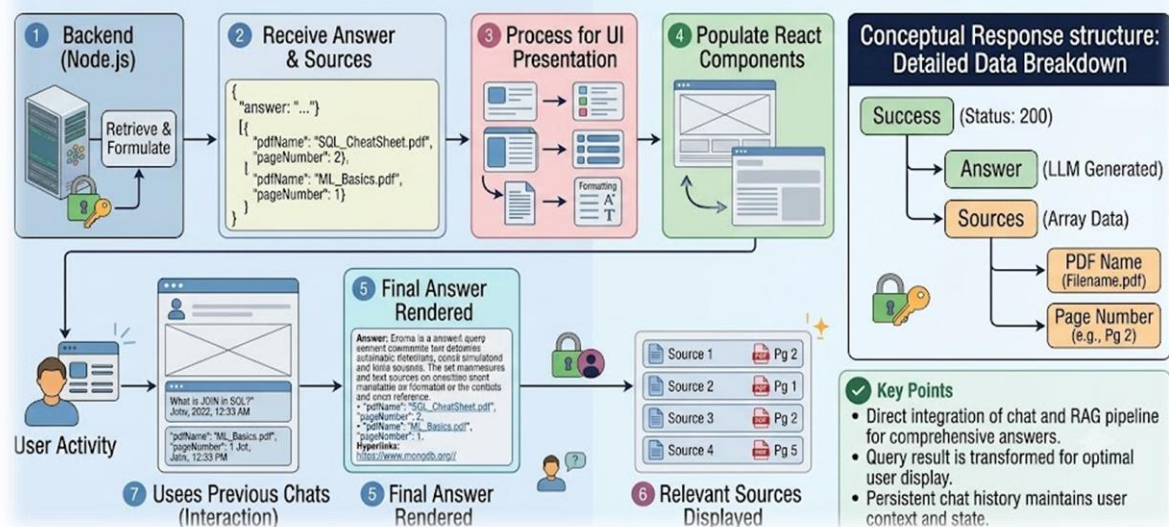


8.10 Final Response to the User

After answer generation, the backend sends the result to the frontend.

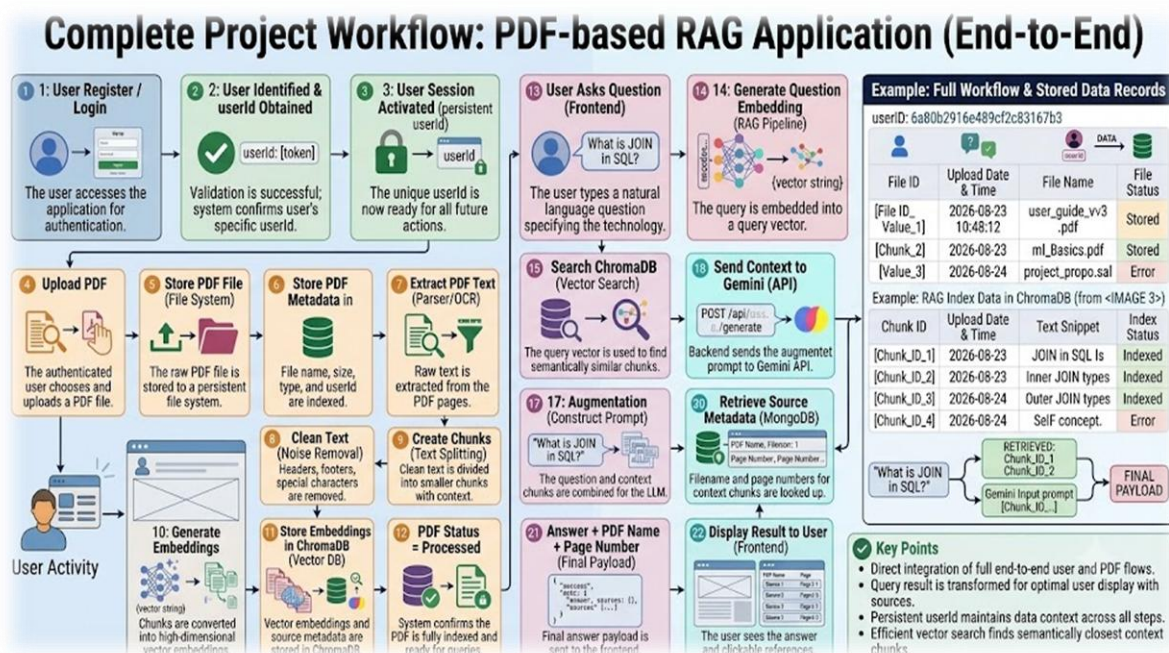
The response contains the generated answer and available source information.

Final Response to the User: Full Workflow & Conceptual Structure



8.11 Complete Project Workflow

The complete end-to-end workflow of the application is:



8.12 Importance of the Complete Workflow

This workflow connects all major parts of the application. MongoDB manages user and PDF metadata, while ChromaDB stores document embeddings for semantic search. The RAG pipeline retrieves relevant information before the question is sent to Gemini.

This design allows the application to manage multiple PDFs, perform semantic search, generate document-based answers, and provide source information to the user.

Section 9 – Advantages and Current Limitations

9.1 Advantages of the Project

The AI-Powered search Knowledge Platform provides a centralized way to manage and search information from multiple PDF documents. Instead of opening different PDFs and manually searching for information, users can upload their documents and ask questions in natural language.

The system uses the RAG approach to retrieve relevant information before generating an answer. This helps the LLM receive relevant document content instead of answering only from its general knowledge.

One important advantage of the system is **semantic search**. The system does not depend only on exact keyword matching. It converts the document content and user question into embeddings and searches for content with similar meaning. Because of this, users can ask questions in different ways and still retrieve relevant information.

The system also supports multiple PDF documents. Each uploaded PDF is processed, divided into chunks, converted into embeddings, and stored in the vector database. This allows the system to search across multiple documents.

MongoDB and ChromaDB have different responsibilities in the project. MongoDB stores user and PDF-related information, while ChromaDB stores document chunks and embeddings for semantic search. This separation makes the data easier to manage.

The project also supports user-wise PDF management through the **userId**. When authentication is fully integrated, the application can automatically identify the logged-in user and show only that user's PDFs and related data.

9.2 Current Limitations

The current version of the project mainly works with text-based PDF documents. If a PDF contains important information only inside images, diagrams, or scanned pages, the system may not understand that information correctly.

The answer quality also depends on the uploaded PDF content. If the required information is not available in the documents, the system cannot retrieve useful context. In such cases, it may return that the answer is not available in the uploaded documents.

The current system is also dependent on the quality of chunking and retrieval. If the document is divided into unsuitable chunks or the relevant chunks are not retrieved, the generated answer may not contain the complete information.

Authentication and automatic user identification are still being integrated. During development and testing, a `userId` may be manually used in some APIs. After the authentication workflow is completed, the backend should automatically get the logged-in user's ID.

The current version also has room for improvement in chat management, PDF management, and retrieval accuracy.

9.3 Future Improvements

In future versions, the project can support image-based PDFs, scanned documents, diagrams, tables, and mathematical formulas. OCR and multimodal AI models can be added to understand information that is not available as normal text.

The retrieval system can also be improved by using hybrid search, which combines keyword search and semantic search. A reranking model can also be added to select the most relevant chunks before sending them to the LLM.

Authentication can be fully integrated so that the system automatically identifies the logged-in user. This will allow proper user-wise management of PDFs and chat history.

Other future improvements can include PDF deletion, document preview, source highlighting, conversation management, multi-language support, and improved chat history.

The project can also be expanded to support a larger number of documents and users.

9.4 Overall Benefit

The main benefit of the project is that it converts a collection of PDF documents into a searchable knowledge system. Users can upload documents, ask questions naturally, retrieve relevant information, and receive answers with source details.

This reduces manual searching time and makes the stored research material easier to access and use.

Conclusion

The AI Search Assistant converts uploaded PDF documents into a searchable knowledge source. MongoDB is used to manage user and PDF metadata, while ChromaDB stores processed document chunks and embeddings for semantic retrieval. The RAG pipeline retrieves relevant document content before Gemini generates the final answer, which helps the system provide answers based on the uploaded documents instead of relying only on general model knowledge. The project provides a strong foundation for document search, user-wise PDF management, semantic retrieval, source information, and future improvements such as complete authentication, chat history, hybrid search, reranking, OCR, multimodal document support, and advanced PDF management.